

Every question, solved and explained

How this is arranged

All **225 questions** from the lecture slides, the reference sheets and both real tests. Grouped by topic, and within each topic ordered **easy first**, so the definitions come before the two-step calculations that assume them.

Every question carries the source it came from. Where the same question was set twice, it appears once and says where else it was asked. The answer is marked, and every other option has a line saying why it fails.

1 The Language, the Toolchain and Objects

1.1

EASY

Manual O.2 Q1

What does `javac` produce, and what runs it?

- a. Machine code for your processor, run directly by the operating system
- b. A `.java` file, run by `javac` itself
- c. An executable `.exe` file, run by double clicking it
- d. Bytecode in a `.class` file, run by the Java Virtual Machine

✗ Machine code for your processor, run directly by the operating system

Wrong. Compiling straight to machine code is the C model, and it is exactly what Java avoids, because machine code runs on one kind of processor only.

✗ A `.java` file, run by `javac` itself

Wrong. The `.java` file is what you WRITE and what `javac` consumes. It is the input, not the output.

✗ An executable `.exe` file, run by double clicking it

Wrong. Java deliberately does not produce a platform executable. That is the whole point of write once, run anywhere.

✓ Bytecode in a `.class` file, run by the Java Virtual Machine

Correct. `javac` turns your `.java` source into bytecode in a `.class` file, and the `java` command starts a JVM that executes that bytecode. The JVM is what makes the same `.class` file run on any machine that has one.

The answer is Bytecode in a `.class` file, run by the Java Virtual Machine. Source to bytecode by `javac`, bytecode to behaviour by the JVM. Java is compiled AND interpreted, and saying only one of those loses half the mark.

Which signature does the JVM look for to start a program?

- a. `public static void main(String[] args)`
- b. `public void main(String[] args)`
- c. `static void Main(String[] args)`
- d. `public static int main(String args)`

✓ `public static void main(String[] args)`

Correct. `public` so the JVM can reach it, `static` so it runs with no object, `void` because it returns nothing, and a `String` array for the command line arguments.

✗ `public void main(String[] args)`

Wrong. Without `static` the JVM would need an object to call it on, and no object exists yet when the program starts.

✗ `static void Main(String[] args)`

Wrong on two counts: Java is case sensitive so `Main` is a different name, and without `public` the JVM cannot reach it.

✗ `public static int main(String args)`

Wrong on two counts: `main` returns `void`, and the parameter is an `ARRAY` of `String`, not a single `String`.

The answer is `public static void main(String[] args)`. `public static void main(String[] args)`. Write it out by hand until it is automatic; a paper answer that gets it wrong has failed before the first statement.

Asked twice. This same question also appears as: Manual O.10 Q3.

Which statement is TRUE?

- a. The JVM contains the JRE, which contains the JDK
- b. The JDK and the JRE are two names for the same thing
- c. The JRE is needed to compile, the JDK only to run
- d. The JDK contains the JRE, which contains the JVM

✗ The JVM contains the JRE, which contains the JDK

Wrong, and it inverts the nesting. The smallest of the three is the JVM.

✗ The JDK and the JRE are two names for the same thing

Wrong. The JRE can only RUN programs. Only the JDK carries javac.

✗ The JRE is needed to compile, the JDK only to run

Wrong, and it swaps them: you compile with the JDK and can run with the JRE alone.

✓ The JDK contains the JRE, which contains the JVM

Correct. The JDK is the developer kit and holds the compiler; inside it is the runtime environment, and inside that is the virtual machine that actually executes bytecode.

The answer is The JDK contains the JRE, which contains the JVM. JDK contains JRE contains JVM.
Compile needs the JDK, run needs only the JRE.

[illegible]

What does this print?

```
System.out.print("A");  
System.out.println("B");  
System.out.print("C");
```

- a. A B
C
- b. AB
C
- c. A
B
C
- d. ABC

~~X~~ A B
C

Wrong. Nothing adds a space between A and B.

✓ AB
C

Correct. `print` stays on the line, `println` ends it after B, and the last `print` starts the new line.

X A

B

C

Wrong. Only println moves to a new line; print does not.

X ABC

Wrong. println("B") ends the line before C.

println prints and then moves to a new line; print leaves the cursor where it is.

1.5

EASY

Lesson 1.1

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
System.out.println("Hi")
```

- a. It does not compile: the println statement has no semicolon
- b. It prints Hi with a warning
- c. It does not compile: the class line needs a semicolon
- d. It prints Hi

✓ It does not compile: the println statement has no semicolon

Correct. Every statement ends in a semicolon, and this one has none.

X It prints Hi with a warning

Wrong. A missing semicolon is an error, not a warning.

X It does not compile: the class line needs a semicolon

Wrong. A line ending in an opening brace is a heading, not a statement, and takes no semicolon.

X It prints Hi

Wrong. The missing semicolon stops compilation.

A missing semicolon is the single most common error in a handwritten script.

Which of these lines needs a semicolon at the end?

- a. `public class Hello {`
- b. `}`
- c. `int total = 0`
- d. `public static void main(String[] args) {`

X `public class Hello {`

Wrong. A line ending in an opening brace opens a block.

X `}`

Wrong. A closing brace ends a block.

✓ `int total = 0`

Correct. An assignment is a statement, and every statement ends in a semicolon.

X `public static void main(String[] args) {`

Wrong. The main line opens a block.

| Semicolons go on statements only: assignments, calls and returns.

A file holds public class ReportCard. What must the file be called?

- a. ReportCard.class
- b. reportcard.java
- c. Main.java
- d. ReportCard.java

✗ ReportCard.class

Wrong. .class is the compiled bytecode, not the source you write.

✗ reportcard.java

Wrong. The case must match: ReportCard.

✗ Main.java

Wrong. The file is named after the class.

✓ ReportCard.java

Correct. A public class lives in a file of exactly its own name.

■ A public class must live in a file named after it, exactly, ending .java.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
system.out.println("hi");
```

- a. It prints hi
- b. It prints hi and a warning
- c. It compiles, then fails when it runs
- d. It does not compile, because Java is case sensitive and the class is System

✗ It prints hi

Wrong. The lower case s stops compilation.

✗ It prints hi and a warning

Wrong. It is an error, not a warning.

✗ It compiles, then fails when it runs

Wrong. The compiler catches it before anything runs.

✓ It does not compile, because Java is case sensitive and the class is System

Correct. system with a small s is not a name Java knows.

String, System and println: case is part of every name.

[illegible]

What does this print?

```
System.out.println("5 + 3");  
System.out.println(5 + 3);
```

a. 8

8

b. 8

 $5 + 3$

c. $5 + 3$

 $5 + 3$

d. $5 + 3$

8

~~X~~ 8

8

Wrong. The quotes make the first line text.

X 8

5

Wrong. The lines are in the other order.

~~X~~ 5 + 3

 $5 + 3$

Wrong. Without quotes, $5 + 3$ is calculated.

✓ $5 + 3$

8

Correct. In quotes it is text and is printed as written; without quotes it is worked out.

Anything in double quotes is a String, printed exactly; an expression outside quotes is evaluated.

1.10 EASY Lesson 1.1

True or false: a comment written after `//` is ignored by the compiler.

Answer: **true**

`//` comments to the end of the line; `/* ... */` comments across several lines. The paper asks for comments in Sections B and C.

1.11 EASY Lesson 1.2

Which file does the JVM actually read and run?

- a. Both files, one after the other
- b. `javac.exe`
- c. `Hello.java`, the source
- d. `Hello.class`, the bytecode

✗ Both files, one after the other

Wrong. Only the bytecode is run.

✗ `javac.exe`

Wrong. `javac` is the compiler that MAKES the `.class` file.

✗ `Hello.java`, the source

Wrong. The JVM never sees your source file.

✓ `Hello.class`, the bytecode

Correct. `javac` turns the source into bytecode in a `.class` file, and the JVM runs that.

Source, compiler, bytecode, virtual machine: `Hello.java`, `javac`, `Hello.class`, the JVM.

1.12**EASY**

Lesson 1.2

What makes Java platform independent?

- a. javac produces machine code for every processor
- b. Every operating system uses the same JVM
- c. The same bytecode runs on any machine that has a JVM
- d. Java has no compiler

X javac produces machine code for every processor

Wrong. javac produces bytecode, not machine code.

X Every operating system uses the same JVM

Wrong. The JVM is the one part that differs from platform to platform.

✓ The same bytecode runs on any machine that has a JVM

Correct. Write once, run anywhere: the bytecode never changes.

X Java has no compiler

Wrong. javac is the compiler.

| The bytecode is the same everywhere, and a platform specific JVM executes it.

1.13**EASY**

Lesson 1.2

True or false: the JVM is the one part of Java that differs from platform to platform.

Answer: true

| Platform independence works because only the JVM changes between machines.

1.14**EASY**

Lesson 1.2

Which feature means memory you stop using is reclaimed for you?

- a. Platform independence
- b. Automatic garbage collection
- c. Multithreading
- d. Strong type checking

✗ Platform independence

Wrong. That is about bytecode running anywhere.

✓ Automatic garbage collection

Correct. Garbage collection frees memory nothing refers to any more.

✗ Multithreading

Wrong. Multithreading is running tasks at the same time.

✗ Strong type checking

Wrong. That is what makes Java robust about types.

Java is simple, secure, robust, multithreaded, and garbage collected.

1.15**EASY**

Lesson 1.2

True or false: `javac` executes the bytecode.

Answer: **false**

Compiler makes it, virtual machine runs it.

Which statement best distinguishes a class from an object?

- a. A class is a running instance and an object is its definition
- b. A class is a blueprint; an object is an instance of it, with its own values
- c. A class and an object are two words for the same thing
- d. A class can have only one object at a time

✗ A class is a running instance and an object is its definition

Wrong, and it is exactly backwards. The definition is the class; the running instance is the object.

✓ A class is a blueprint; an object is an instance of it, with its own values

Correct. The class says what every instance will have and be able to do; each object made with new has its own copy of the fields.

✗ A class and an object are two words for the same thing

Wrong. If they were the same, new would have nothing to do.

✗ A class can have only one object at a time

Wrong. A class exists so that many objects can be created from it.

The answer is A class is a blueprint; an object is an instance of it, with its own values. Class is the blueprint, object is the building. new is what turns one into the other.

Java is described as an object-oriented language. Which TWO features below are the reason? Choose the single option naming BOTH correctly.

- a. Pointers and manual memory management
- b. Preprocessor directives and macros
- c. Encapsulation and inheritance
- d. Goto statements and labels

X Pointers and manual memory management

Wrong. Java deliberately has neither: references replace pointers and the garbage collector replaces manual freeing.

X Preprocessor directives and macros

Wrong. Those are C and C++ features. Java has no preprocessor at all.

✓ Encapsulation and inheritance

Correct. Bundling data with the methods that act on it, and letting one class extend another, are two of the four pillars, alongside abstraction and polymorphism.

X Goto statements and labels

Wrong. goto is a reserved word in Java that is not usable, kept reserved precisely so nobody writes one.

The answer is Encapsulation and inheritance. Encapsulation, inheritance, polymorphism, abstraction. Those four are the answer to any what-makes-it-object-oriented question.

A program compiles without error but refuses to start. Which main line explains it?

- a. `static public void main(String[] args)`
- b. `public static void main(String args[])`
- c. `public static void main(String[] args)`
- d. `public static void Main(String[] args)`

✗ `static public void main(String[] args)`

Wrong. The order of public and static does not matter.

✗ `public static void main(String args[])`

Wrong. `String args[]` is another legal way to declare the same array.

✗ `public static void main(String[] args)`

Wrong. That is the correct header.

✓ `public static void Main(String[] args)`

Correct. Main with a capital M is a different name, so there is no starting point, though it compiles.

Java is case sensitive. `main` is the name the JVM looks for; `Main` compiles and never runs.

How do the JDK, the JRE and the JVM fit together?

- a. The JVM contains the JRE, which contains the JDK
- b. The JRE contains the JDK and the JVM
- c. They are three unrelated programs
- d. The JDK contains the JRE, which contains the JVM

✗ The JVM contains the JRE, which contains the JDK

Wrong. This is upside down.

✗ The JRE contains the JDK and the JVM

Wrong. The JDK is the largest.

✗ They are three unrelated programs

Wrong. They nest.

✓ The JDK contains the JRE, which contains the JVM

Correct. Each adds to the one inside it: JDK adds javac, JRE adds the class libraries, JVM executes.

Draw three nested boxes: JDK around JRE around JVM.

Which two statements are true?

- a. The JDK adds development tools, principally javac
- b. The JVM compiles your source code
- c. The JVM reads your .java file
- d. The JRE is the JVM plus the standard class libraries

✓ The JDK adds development tools, principally javac

Correct. That is the JDK.

✗ The JVM compiles your source code

Wrong. javac compiles; the JVM executes.

✗ The JVM reads your .java file

Wrong. The JVM only ever sees the .class bytecode.

✓ The JRE is the JVM plus the standard class libraries

Correct. That is the JRE.

javac compiles to bytecode, the JVM runs bytecode. Getting those two the wrong way round is the commonest lost mark here.

A Java file has a syntax error on line 40. How much of the program runs?

- a. Lines 1 to 39
- b. Everything except line 40
- c. None of it; the compiler rejects the file before anything runs
- d. Everything, with a warning at line 40

✗ Lines 1 to 39

Wrong. That is how an interpreted language behaves with a runtime error.

✗ Everything except line 40

Wrong. A file that does not compile does not run at all.

✓ None of it; the compiler rejects the file before anything runs

Correct. Java is compiled, so the whole file is checked first.

✗ Everything, with a warning at line 40

Wrong. A syntax error is not a warning.

| A syntax error means nothing runs, not even the correct lines above it.

2 Types, Operators, Input and Output

2.1

EASY

Manual O.3 Q10

A program reads an integer with a Scanner named in. Which line is correct?

- a. `int n = in.nextInt();`
- b. `int n = in.next();`
- c. `int n = in.nextLine();`
- d. `int n = Scanner.nextInt();`

✓ `int n = in.nextInt();`

Correct. `nextInt()` reads the next token and returns it as an int, which is what the variable needs.

✗ `int n = in.next();`

Wrong. `next()` returns a String, and a String cannot be assigned to an int without conversion.

✗ `int n = in.nextLine();`

Wrong for the same reason: `nextLine()` also returns a String. It would need `Integer.parseInt` around it.

✗ `int n = Scanner.nextInt();`

Wrong. `nextInt` is an instance method, called on your Scanner object, not on the class.

The answer is `int n = in.nextInt();`. `nextInt` for an int, `nextDouble` for a double, `nextLine` for a whole line of text. Mixing `nextInt` with `nextLine` leaves the newline behind, which is the classic Scanner bug.

Which assignment compiles WITHOUT a cast?

- a. `int n = 3.5;`
- b. `int n = 7L;`
- c. `double d = 7;`
- d. `char c = "A";`

X `int n = 3.5;`

Wrong. A double into an int loses information, so Java requires you to say so with an explicit cast.

X `int n = 7L;`

Wrong. long to int is narrowing, so it needs a cast.

✓ `double d = 7;`

Correct. int to double is a WIDENING conversion: nothing can be lost, so Java does it silently.

X `char c = "A";`

Wrong. Double quotes make a String, and a String is not a char. A char literal uses single quotes.

The answer is `double d = 7;`. Widening is automatic, narrowing needs a cast. Remember the direction: small into big is safe, big into small is your responsibility.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
char grade = "A";  
System.out.println(grade);
```

- a. It prints 65
- b. It compiles and prints nothing
- c. It does not compile: "A" is a String, which does not fit in a char
- d. It prints A

✗ It prints 65

Wrong. It never compiles, so nothing prints.

✗ It compiles and prints nothing

Wrong. The type mismatch stops compilation.

✓ It does not compile: "A" is a String, which does not fit in a char

Correct. A char takes single quotes, 'A'.

✗ It prints A

Wrong. The double quotes make it a String.

Double quotes for a String, single quotes for a char. A value that does not fit its box is the planted error to look for.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
double d = 7;  
System.out.println(d);
```

- a. 7.0
- b. 7
- c. 7.00
- d. It does not compile

✓ 7.0

Correct. An int widens to a double automatically, and a double prints with a decimal point.

✗ 7

Wrong. d is a double, so it prints as 7.0.

✗ 7.00

Wrong. println shows 7.0.

✗ It does not compile

Wrong. Widening is always allowed.

Widening is free: byte to short to int to long to float to double. Narrowing needs a cast.

Which two of these are legal variable names?

- a. 2price
- b. price2
- c. class
- d. _total

✗ 2price

Wrong. A name may not start with a digit.

✓ price2

Correct. Digits are allowed after the first character.

✗ class

Wrong. class is a Java keyword.

✓ _total

Correct. A name may start with an underscore.

Letters, digits, _ and \$; not starting with a digit; no spaces; not a keyword.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
final int MAX = 5;  
MAX = 6;  
System.out.println(MAX);
```

- a. It prints 6
- b. It prints 11
- c. It does not compile: a final variable cannot be assigned again
- d. It prints 5

X It prints 6

Wrong. The reassignment is refused.

X It prints 11

Wrong. It never runs.

✓ It does not compile: a final variable cannot be assigned again

Correct. final means the value can never change after it is set.

X It prints 5

Wrong. It never runs.

| final makes a constant, named in capitals: MAX, VAT.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int x = 10;  
long y = x;  
System.out.println(y + 1);
```

- a. 101
- b. 11
- c. 11.0
- d. It does not compile

X 101

Wrong. + adds numbers; nothing here is text.

✓ 11

Correct. An int widens to a long automatically, and a long plus 1 is the whole number 11.

X 11.0

Wrong. long is a whole-number type.

X It does not compile

Wrong. int to long is widening, which is always allowed.

| Moving to a bigger box is silent and safe.

2.8

EASY

Lesson 1.3

True or false: `String` is one of Java's eight primitive types.

Answer: **false**

Every primitive is lower case; `String`, with its capital `S`, is a class.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
System.out.println(29 / 4 + " " + 29 % 4);
```

- a. 7.25 1
- b. 8 1
- c. 7 0.25
- d. 7 1

X 7.25 1

Wrong. Two ints divide as ints and the remainder is dropped.

X 8 1

Wrong. Integer division does not round up.

X 7 0.25

Wrong. % gives the remainder as a whole number, 1.

✓ 7 1

Correct. 29 / 4 between ints is 7, remainder 1.

| Two ints divide as ints. The remainder is what % gives you.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
int x = 5;
if (x = 5) {
    System.out.println("five");
}
```

- a. It prints nothing
- b. It prints five, then x is 5
- c. It prints five
- d. It does not compile: if needs a boolean, and x = 5 is an assignment

✗ It prints nothing

Wrong. It never compiles.

✗ It prints five, then x is 5

Wrong. It never compiles.

✗ It prints five

Wrong. It never compiles.

✓ It does not compile: if needs a boolean, and x = 5 is an assignment

Correct. = assigns; == compares. Java refuses an int where if needs a boolean.

Writing if (x = 5) will not compile in Java, a small mercy.

Which import does a JOptionPane program need?

- a. `import java.util.JOptionPane;`
- b. `import javax.swing.JOptionPane;`
- c. `import java.swing.JOptionPane;`
- d. `import javax.util.Scanner;`

✗ `import java.util.JOptionPane;`
Wrong. `java.util` is Scanner's package.

✓ `import javax.swing.JOptionPane;`
Correct. `javax`, with the x, then `swing`.

✗ `import java.swing.JOptionPane;`
Wrong. It is `javax`, not `java`.

✗ `import javax.util.Scanner;`
Wrong. Scanner is `java.util.Scanner`.

| `java.util.Scanner`, `javax.swing.JOptionPane`: one letter, whole import.

The user types 42 into a `JOptionPane.showInputDialog`. What does the call return?

- a. The int 42
- b. Nothing; it only shows a window
- c. The String "42"
- d. The double 42.0

✗ The int 42

Wrong. You must convert it with `Integer.parseInt`.

✗ Nothing; it only shows a window

Wrong. `showInputDialog` returns what was typed.

✓ The String "42"

Correct. A dialog only ever hands back text.

✗ The double 42.0

Wrong. It is text until you parse it.

! A dialog always gives you text, so parse it: `Integer.parseInt(t)` or `Double.parseDouble(t)`.

Which line should end a JOptionPane program, so it stops when main ends?

- a. `System.out.println();`
- b. `System.exit(0);`
- c. `JOptionPane.close();`
- d. `return 0;`

✗ `System.out.println();`

Wrong. That only prints a blank line.

✓ `System.exit(0);`

Correct. A window program does not stop on its own; `System.exit(0)` ends it.

✗ `JOptionPane.close();`

Wrong. There is no such call.

✗ `return 0;`

Wrong. `main` is void, so it returns no value.

Finish a dialog program with `System.exit(0)`.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
System.out.printf("%d items at %.2f%n", 3, 2.5);
```

- a. %d items at %.2f
- b. 3.00 items at 2.50
- c. 3 items at 2.5
- d. 3 items at 2.50

X %d items at %.2f

Wrong. The placeholders are replaced by the values.

X 3.00 items at 2.50

Wrong. %d is a whole number.

X 3 items at 2.5

Wrong. %.2f always shows two decimal places.

✓ 3 items at 2.50

Correct. %d takes the whole number 3 and %.2f shows 2.5 to two decimal places.

| %d for whole numbers, %.2f for money, %s for text, %n for the new line.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
System.out.println((int) 7.89);  
System.out.println(Math.round(7.89));
```

a. 7

7

b. 8

8

c. 7

8

d. 7.89

8

X 7

7

Wrong. Math.round(7.89) rounds up to 8.

X 8

8

Wrong. The cast does not round.

✓ 7

8

Correct. A cast to int chops off the decimal part; Math.round rounds to the nearest.

✗ 7.89

8

Wrong. (int) produces a whole number.

| (int) 7.89 is 7: a cast chops. For proper rounding use Math.round.

2.16

MEDIUM

Lesson 1.3

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
float f = 1.5;  
System.out.println(f);
```

- a. It prints 1.5
- b. It prints 1.0
- c. It does not compile: 1.5 is a double; write 1.5f
- d. It prints 2

✗ It prints 1.5

Wrong. The literal does not fit the float box as written.

✗ It prints 1.0

Wrong. It does not compile.

✓ It does not compile: 1.5 is a double; write 1.5f

Correct. A decimal literal is a double, and a double does not fit a float without the f suffix.

✗ It prints 2

Wrong. It does not compile.

| Two suffixes to remember: a long literal ends in L, a float literal ends in f.

2.17

MEDIUM

Lesson 1.3

How many bytes does a char take in Java?

- a. 4
- b. 2
- c. 1
- d. It is not defined

✗ 4

Wrong. Four bytes is an int or a float.

✓ 2

Correct. Java characters are Unicode, two bytes.

✗ 1

Wrong. One byte is a byte.

✗ It is not defined

Wrong. It is boolean whose size is not defined.

byte 1, short 2, int 4, long 8, float 4, double 8, char 2, boolean undefined.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
System.out.println("x" + 2 + 3);  
System.out.println(2 + 3 + "x");
```

- a. x5
5x
- b. x23
5x
- c. x5
23x
- d. x23
23x

X x5
5x

Wrong. Once text is on the left, every later + joins.

✓ x23
5x

Correct. + works left to right: "x" + 2 is already text, so 3 joins too; 2 + 3 are both numbers, so they add before meeting "x".

X x5
23x

Wrong. Both lines are the other way round.

X x23
23x

Wrong. 2 + 3 adds before the text is reached.

| If either side of + is a String, + joins. Bracket arithmetic that sits next to text.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int n = 0;
if (n != 0 && 10 / n > 1) {
    System.out.println("big");
} else {
    System.out.println("safe");
}
```

- a. big
- b. safe
- c. an ArithmeticException
- d. It does not compile

X big

Wrong. The condition is false.

✓ safe

Correct. `n != 0` is false, so `&&` stops and the division is never attempted.

X an ArithmeticException

Wrong. The guard on the left prevents the division by zero.

X It does not compile

Wrong. It compiles.

| `&&` stops at the first false, so put the guard on the left: `n != 0 && total / n > 1`.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
System.out.println(10 - 2 * 3 + 4 / 2);
```

- a. 6
- b. -2
- c. 14
- d. 8

✓ 6

Correct. * and / first: $2 * 3 = 6$ and $4 / 2 = 2$; then $10 - 6 + 2 = 6$.

✗ -2

Wrong. The division gives 2, which is added.

✗ 14

Wrong. That works left to right ignoring precedence.

✗ 8

Wrong. Check the subtraction: $10 - 6$ is 4, plus 2 is 6.

Precedence: brackets, unary and casts, * / %, then + -, left to right.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int x = 5;  
x += 3;  
x *= 2;  
x %= 5;  
System.out.println(x);
```

- a. 0
- b. 3
- c. 16
- d. 1

X 0

Wrong. 16 is not a multiple of 5.

X 3

Wrong. Work each line in turn: 8, 16, 1.

X 16

Wrong. The last line takes the remainder.

✓ 1

Correct. $5 + 3 = 8$, times 2 is 16, and $16 \% 5$ leaves 1.

$x += 3$ means $x = x + 3$; the same for $*=$ and $\%=$.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
String t = "42";  
int n = Integer.parseInt(t);  
System.out.println(n + 1);  
System.out.println(t + 1);
```

- a. 421
421
- b. 43
421
- c. 43
43
- d. It does not compile

X 421
421

Wrong. n is a number after parseInt.

✓ 43
421

Correct. n is the number 42, so n + 1 is 43; t is still the text "42", so t + 1 joins to 421.

✗ 43

43

Wrong. t was never converted.

✗ It does not compile

Wrong. Both lines are legal.

Integer.parseInt turns text into a whole number; the original String is unchanged.

2.23

MEDIUM

Lesson 1.5

Asked twice. This same question also appears as: Lesson 2.2 and Lesson 2.8 and Lesson 3.1 and Lesson 3.5 and Lesson 3.6.

What happens when this runs?

```
int n = Integer.parseInt("4.5");  
System.out.println(n);
```

- a. It prints 5
- b. It prints 4.5
- c. It throws NumberFormatException, because 4.5 is not a whole number
- d. It prints 4

✗ It prints 5

Wrong. parseInt does not round.

✗ It prints 4.5

Wrong. That needs Double.parseDouble.

✓ It throws NumberFormatException, because 4.5 is not a whole number

Correct. parseInt accepts only whole-number text.

✗ It prints 4

Wrong. parseInt does not chop; it refuses.

Integer.parseInt("4.5") fails; Double.parseDouble("4.5") gives 4.5.

Asked twice. This same question also appears as: Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q301 {  
    public static void main(String[] args) {  
        System.out.println(7 / 2);  
        System.out.println(7 % 2);  
        System.out.println(7.0 / 2);  
    }  
}
```

a. 3.5

1

3.5

b. 3

1

3

c. 3

1

3.5

d. 3.5

3.5

3.5

✗ 3.5

1

3.5

Wrong on the first line. Integer division truncates in Java; it does not produce a decimal. That is the single most common Java arithmetic error.

✗ 3

1

3

Wrong on the last line. 7.0 is a double, so the division is done in doubles.

✓ 3

1

3.5

Correct. Two ints divide as ints, so the fraction is thrown away and $7 / 2$ is 3. The remainder is 1. Making either operand a double promotes the whole expression, so $7.0 / 2$ is 3.5.

✗ 3.5

3.5

3.5

Wrong throughout. % is the remainder operator, not a division.

The answer is 3 then 1 then 3.5. int divided by int gives int, and the fraction is discarded, not rounded. One double operand promotes the whole expression.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q302 {  
    public static void main(String[] args) {  
        System.out.println(-7 / 2);  
        System.out.println(-7 % 2);  
    }  
}
```

a. -4

1

b. -4

-1

c. -3

1

d. -3

-1

X -4

1

Wrong. Java truncates towards ZERO, so $-7 / 2$ is -3, not -4. Flooring to -4 is what Python does, and the difference is a real exam trap.

X -4

-1

Wrong on the division for the same reason: Java truncates towards zero.

X -3

1

Wrong on the remainder. In Java the result of % has the sign of the dividend, so a negative dividend gives a negative remainder.

✓ -3

-1

Correct. Division truncates towards zero giving -3, and the remainder takes the sign of the LEFT operand, giving -1.

The answer is -3 then -1. Java truncates towards zero and % takes the sign of the left operand. Both differ from some other languages, so this is worth knowing exactly rather than by feel.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q303 {  
    public static void main(String[] args) {  
        char c = 'A';  
        System.out.println(c + 1);  
        System.out.println((char) (c + 1));  
    }  
}
```

a. 66

B

b. B

B

c. A1

B

d. 66

66

✓ 66

B

Correct. `c + 1` promotes the char to its code, 65, and adds 1 to give 66. The cast on the second line turns 66 back into a char, which prints as B.

X B

B

Wrong on the first line. A char in arithmetic is promoted to int, so the result is a number, not a letter.

X A1

B

Wrong. Plus between a char and an int is ARITHMETIC, not joining. Joining would need one side to be a String.

X 66

66

Wrong on the second line: the cast to char is what makes it print as a letter.

The answer is 66 then B. A char is a number with a costume on. Arithmetic strips the costume; a cast back to char puts it on again.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q304 {  
    public static void main(String[] args) {  
        System.out.println("Total: " + 1 + 2);  
        System.out.println("Total: " + (1 + 2));  
    }  
}
```

- a. Total: 3
Total: 3
- b. Total: 12
Total: 3
- c. Total: 12
Total: 12
- d. Total: 3
Total: 12

X Total: 3
Total: 3

Wrong on the first line. Nothing adds 1 and 2 there: by the time the 2 is reached the left side is already a String.

✓ Total: 12

Total: 3

Correct. Plus groups left to right. The first plus has a String on its left, so it JOINS, producing "Total: 1", and the next plus joins the 2 onto that. The brackets on the second line force the addition first.

✗ Total: 12

Total: 12

Wrong on the second line. The brackets change the answer, which is the whole point of the question.

✗ Total: 3

Total: 12

Wrong, and it has the two lines swapped.

The answer is Total: 12 then Total: 3. Plus is left-associative, and once a String is involved every later plus joins. Bracket the arithmetic and this whole class of bug goes away.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q305 {  
    public static void main(String[] args) {  
        int big = Integer.MAX_VALUE;  
        System.out.println(big);  
        System.out.println(big + 1);  
    }  
}
```

- a. 2147483647
2147483648
- b. 2147483647
error at run time
- c. 2147483647
-2147483648
- d. 2147483648
-2147483648

X 2147483647
2147483648

Wrong. An int cannot hold 2147483648. The type does not grow to fit.

✗ 2147483647

error at run time

Wrong. There is no exception for integer overflow in Java. That silence is exactly what makes it dangerous.

✓ 2147483647

-2147483648

Correct. int is 32 bits, so its maximum is 2147483647, and adding 1 wraps round to the most negative value. Java does not report this: it just gives the wrong number quietly.

✗ 2147483648

-2147483648

Wrong on the first line. MAX_VALUE is 2147483647, one less than 2 to the power 31.

The answer is 2147483647 then -2147483648. int holds -2147483648 to 2147483647, and overflow wraps silently. Use long when a value might exceed it.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q306 {  
    public static void main(String[] args) {  
        double a = 0.1 + 0.2;  
        System.out.println(a);  
        System.out.println(a == 0.3);  
    }  
}
```

- a. 0.3
true
- b. 0.3
false
- c. 0.30000000000000004
true
- d. 0.30000000000000004
false

X 0.3
true

Wrong on both lines. Binary floating point cannot hold 0.1 or 0.2 exactly, so their sum is not exactly 0.3.

✗ 0.3
false

Wrong on the first line: println shows the true stored value, tiny error and all.

✗ 0.30000000000000004
true

Wrong on the second line: if the value printed is not 0.3, it cannot equal 0.3.

✓ 0.30000000000000004
false

Correct. The stored sum is very slightly above 0.3 and prints in full, so the equality test is false.

The answer is 0.30000000000000004 then false. Never compare two doubles with ==. Compare the size of their difference against a small tolerance instead.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q307 {  
    public static void main(String[] args) {  
        int i = 5;  
        System.out.println(i++);  
        System.out.println(i);  
        System.out.println(++i);  
    }  
}
```

a. 5

6

7

b. 6

6

7

c. 5

5

6

d. 6

7

7

✓ 5
6
7

Correct. `i++` yields the OLD value 5 and then makes `i` 6. The second line shows that 6. `++i` increments first, to 7, and yields 7.

✗ 6
6
7

Wrong on the first line. The post-increment gives back the value BEFORE the increment.

✗ 5
5
6

Wrong on the second line. The increment did happen, so `i` is 6 by then.

✗ 6
7
7

Wrong on the first line, for the same reason as 6, 6, 7: the post-increment gives back the value BEFORE the increment, so the first line is 5.

The answer is 5 then 6 then 7. Post says use it then change it; pre says change it then use it. The variable ends up the same either way, and what the EXPRESSION gives back is what differs.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q308 {  
    public static void main(String[] args) {  
        System.out.println(2 + 3 * 4);  
        System.out.println((2 + 3) * 4);  
        System.out.println(10 - 2 - 3);  
    }  
}
```

a. 20

20

5

b. 14

20

5

c. 14

20

11

d. 14

14

5

✗ 20

20

5

Wrong on the first line. Multiplication binds tighter than addition without any brackets.

✓ 14

20

5

Correct. Multiply before add gives 14, brackets force 20, and subtraction groups left to right so $10 - 2$ is 8 and $8 - 3$ is 5.

✗ 14

20

11

Wrong on the last line. Left to right gives 5; grouping the right pair first would give 11, and no operator in Java does that.

✗ 14

14

5

Wrong on the second line: the brackets really do change it.

The answer is 14 then 20 then 5. Multiply and divide before add and subtract, then left to right. When in doubt, write the brackets yourself.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q309 {
    static boolean shout(String s) {
        System.out.println(s);
        return true;
    }
    public static void main(String[] args) {
        if (false && shout("left")) { }
        if (true || shout("right")) { }
        System.out.println("done");
    }
}
```

- a. left
right
done
- b. left
done
- c. done
- d. right
done

X left
right

done

Wrong. Neither call happens, which is the whole point of short circuiting.

✗ left

done

Wrong. false && anything is already decided, so the right side is skipped.

✓ done

Correct. && stops as soon as its left side is false, and || stops as soon as its left side is true, so neither shout is ever called.

✗ right

done

Wrong. true || anything is already decided, so the right side is skipped.

The answer is done. && and || evaluate the right side only when they must. That is what makes `if (s != null && s.length() > 0)` safe.

HARD

Manual O.3 Q11

[illegible]

What does this print?

```
public class Q311 {
    public static void main(String[] args) {
        System.out.printf("%.2f%n", 3.14159);
        System.out.printf("%5d|%n", 42);
    }
}
```

- a. 3.142
42 |
- b. 3.14
42|
- c. 3.14
42 |
- d. 3.1
42|

X 3.142
42 |

Wrong on both counts: .2f is two decimals, and %5d right-aligns.

✓ 3.14
42|

Correct. `%.2f` gives two decimal places, and `%5d` pads the number to five characters on the LEFT, so the spaces come first.

X 3.14
42 |

Wrong on the padding. A width without a minus sign right-aligns; `%-5d` would pad on the right.

X 3.1
42|

Wrong on the decimals: two were asked for, not one.

The answer is 3.14 then 42|. `%.2f` fixes the decimals, `%5d` fixes the width and right-aligns, `%-5d` left-aligns, and `%n` is the newline that works everywhere.

[illegible]

What does this print?

```
int sum = 7, n = 2;
System.out.println(sum / n);
System.out.println((double) sum / n);
System.out.println((double) (sum / n));
```

- a. 3.5
3.5
3.5
- b. 3
3
3
- c. 3
3.5
3.5
- d. 3
3.5
3.0

✗ 3.5

3.5

3.5

Wrong. The first line divides two ints.

✗ 3

3

3

Wrong. The second line casts sum before dividing.

✗ 3

3.5

3.5

Wrong. `(double)(sum / n)` divides first, in integers.

✓ 3

3.5

3.0

Correct. `sum / n` is integer division, 3; casting one side first gives 3.5; casting AFTER the division only converts the wrong 3 into 3.0.

Cast one operand, before the division: `(double) sum / n`. `(double)(sum / n)` converts an answer that is already wrong.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int a = 3;
int b = a++ + ++a;
System.out.println(a + " " + b);
```

a. 5 7

b. 4 8

c. 5 8

d. 5 9

X 5 7

Wrong. `++a` increments to 5 before it is used.

X 4 8

Wrong. Both increments happen, so `a` ends at 5.

✓ 5 8

Correct. `a++` gives 3 then makes `a` 4; `++a` makes `a` 5 and gives 5; so `b` is $3 + 5 = 8$ and `a` is 5.

X 5 9

Wrong. `a++` contributes its OLD value, 3.

| Post uses the old value then adds one; pre adds one then uses the new value.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

- a. 339
- b. 15
- c. 3345
- d. 12345

X 339

Wrong. 4 and 5 are joined, not added, after the text.

X 15

Wrong. Once text appears, the rest joins.

✓ 3345

Correct. 1 + 2 adds to 3; then "3" makes it text, so 4 and 5 are joined on: 3, 3, 4, 5.

X 12345

Wrong. The first two numbers add before the text.

| Left to right: numbers add until a String appears, then everything joins.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
System.out.printf("%.1f|%5d|%4s|\n", 2.46, 42, "ab");
```

a. 2.4| 42|ab |

b. 2.46|42|ab|

c. 2.5| 42|ab |

d. 2.5|42 | ab|

✗ 2.4| 42|ab |

Wrong. %.1f rounds 2.46 to 2.5.

✗ 2.46|42|ab|

Wrong. The widths and the precision change the output.

✓ 2.5| 42|ab |

Correct. %.1f rounds to one place, %5d pads 42 to five columns on the left, %-4s pads ab to four on the right.

✗ 2.5|42 | ab|

Wrong. Numbers pad on the left; the minus makes text pad on the right.

% starts a placeholder, the number sets width and decimals, the letter sets the type; %n ends the line.

The user types 5, presses Enter, then types Ada. What does this print?

```
Scanner in = new Scanner(System.in);  
int n = in.nextInt();  
String name = in.nextLine();  
System.out.println "[" + name + "];
```

- a. [5 Ada]
- b. [5]
- c. []
- d. [Ada]

✗ [5 Ada]

Wrong. `nextLine` reads only the rest of the current line.

✗ [5]

Wrong. `nextInt` already consumed the 5.

✓ []

Correct. `nextInt` reads 5 but leaves the newline behind, so `nextLine` returns the empty rest of that line.

✗ [Ada]

Wrong. `nextLine` never reaches Ada; it stops at the leftover newline.

Reading a number and then a line: `nextInt` leaves the newline, so add an extra `nextLine()` to swallow it.

3 Control Structures

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int n = 90210, d = 0;
while (n > 0) {
    d++;
    n /= 10;
}
System.out.println(d);
```

- a. 5
- b. 90210
- c. 4
- d. 6

✓ 5

Correct. Five divisions by 10 bring 90210 to 0: 9021, 902, 90, 9, 0.

✗ 90210

Wrong. d counts; n is used up.

✗ 4

Wrong. The zeros inside the number still count.

X 6

Wrong. The loop stops as soon as n reaches 0.

A counting loop destroys what it counts, which is why a program copies n into $temp$ first.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int a = 7, b = 9;  
int m = (a > b) ? a : b;  
System.out.println(m);
```

- a. 9
- b. 16
- c. true
- d. 7

✓ 9

Correct. $a > b$ is false, so the ternary gives the value after the colon, b.

✗ 16

Wrong. Nothing adds a and b.

✗ true

Wrong. The ternary gives one of the two values, not the test.

✗ 7

Wrong. The first value is chosen only when the test is true.

test ? valueIfTrue : valueIfFalse is an if-else squeezed into an expression.

Why does the dry-run table include the pass where the loop test FAILS?

- a. The body runs once more on that pass
- b. It prints an extra line
- c. The examiner does not want it
- d. It shows why the loop stopped, which is where off-by-one answers are caught

✗ The body runs once more on that pass

Wrong. The body does not run when the test fails.

✗ It prints an extra line

Wrong. Nothing is printed on that pass.

✗ The examiner does not want it

Wrong. Writing it is the habit that catches the error.

✓ It shows why the loop stopped, which is where off-by-one answers are caught

Correct. The failing test decides the answer between $<$ and $<=$.

| A table, one row per pass, including the pass that fails the test.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int score = 45;
char grade;
if (score >= 70) {
    grade = 'A';
} else if (score >= 60) {
    grade = 'B';
} else if (score >= 50) {
    grade = 'C';
} else if (score >= 45) {
    grade = 'D';
} else {
    grade = 'F';
}
System.out.println("Grade: " + grade);
```

- a. Grade: B
- b. Grade: C
- c. Grade: D
- d. Grade: F

X Grade: B

Wrong. 45 is far below 60.

X Grade: C

Wrong. 45 is not ≥ 50 .

✓ Grade: D

Correct. 45 fails 70, 60 and 50, then matches 45, and the ladder stops.

✗ Grade: F

Wrong. $45 \geq 45$ is true, so the else is never reached.

A ladder runs exactly one block: the first whose condition is true.

3.5

EASY Lesson 2.1

Which Java condition accepts a score from 0 to 100 inclusive?

- a. $0 \leq \text{score} \leq 100$
- b. $\text{score} > 0 \ \&\& \ \text{score} < 100$
- c. $\text{score} \geq 0 \ \&\& \ \text{score} \leq 100$
- d. $\text{score} \geq 0 \ || \ \text{score} \leq 100$

✗ $0 \leq \text{score} \leq 100$

Wrong. Java cannot chain comparisons; that does not compile.

✗ $\text{score} > 0 \ \&\& \ \text{score} < 100$

Wrong. That excludes 0 and 100.

✓ $\text{score} \geq 0 \ \&\& \ \text{score} \leq 100$

Correct. Both comparisons, joined with $\&\&$.

✗ $\text{score} \geq 0 \ || \ \text{score} \leq 100$

Wrong. $||$ is true for every number.

Java has no $0 \leq \text{score} \leq 100$: write both comparisons and join them.

Which control structure do if and switch belong to?

- a. Recursion
- b. Sequence
- c. Selection
- d. Iteration

✗ Recursion

Wrong. Not one of the three control structures.

✗ Sequence

Wrong. Sequence is statements one after another.

✓ Selection

Correct. Selection chooses between alternatives so that only one runs.

✗ Iteration

Wrong. Iteration is for, while and do-while.

Sequence, selection, iteration.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int month = 1;
String season;
switch (month) {
    case 12: case 1: case 2:
        season = "Harmattan";
        break;
    case 6: case 7: case 8:
        season = "Rainy";
        break;
    default:
        season = "Other";
}
System.out.println(season);
```

- a. Other
- b. Harmattan
- c. Rainy
- d. It does not compile

X Other

Wrong. Month 1 matches a case.

✓ Harmattan

Correct. Cases 12 and 1 fall into case 2, which sets Harmattan and breaks.

✗ Rainy

Wrong. Month 1 is in the first group.

✗ It does not compile

Wrong. Empty cases sharing an answer are legal.

Deliberate fall-through is how several values share one answer.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int day = 9;
double price;
switch (day) {
    case 1: price = 75.0; break;
    case 2: price = 60.0; break;
    case 3: price = 45.0; break;
    default: price = 0.0;
}
System.out.println("Price " + price);
```

- a. Nothing
- b. It does not compile
- c. Price 45.0
- d. Price 0.0

X Nothing

Wrong. The println after the switch always runs.

X It does not compile

Wrong. It compiles.

X Price 45.0

Wrong. Case 3 needs day to be 3.

✓ Price 0.0

Correct. 9 matches no case, so default sets the price to 0.0.

| default catches everything else. Always write one.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
String c = "B";
switch (c) {
    case "A": System.out.println("Excellent"); break;
    case "B": System.out.println("Good"); break;
    default: System.out.println("Fail");
}
```

- a. Excellent
Good
- b. It does not compile
- c. Good
- d. Fail

✗ Excellent
Good

Wrong. It jumps straight to the matching case.

✗ It does not compile

Wrong. Switching on a String is allowed.

✓ Good

Correct. A switch can take a String and matches "B" exactly.

X Fail

Wrong. "B" has its own case.

| A switch compares one value against exact constants, and a String is allowed.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int code = 7;
String label = "none";
switch (code) {
    case 1: label = "one"; break;
    case 2: label = "two"; break;
}
System.out.println(label);
```

- a. Nothing at all
- b. two
- c. one
- d. none

X Nothing at all

Wrong. The println after the switch still runs.

X two

Wrong. code is 7.

X one

Wrong. code is 7.

✓ none

Correct. Nothing matches and there is no default, so label keeps its starting value.

Without a default, an unmatched value does nothing at all.

3.11

EASY

Lesson 2.2

Grading a score into bands, 70 and above, 60 and above, and so on. Which structure?

- a. A do-while loop
- b. A switch with a case for every possible score
- c. An if-else ladder, because each band is a range
- d. A switch, because there are several grades

✗ A do-while loop

Wrong. Nothing here repeats.

✗ A switch with a case for every possible score

Wrong. Possible, but absurd, and not what the paper expects.

✓ An if-else ladder, because each band is a range

Correct. A case must be an exact value; bands are ranges.

✗ A switch, because there are several grades

Wrong. A switch cannot test a range.

Switch for exact values, ladder for ranges.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int n = 10;
while (n < 5) {
    System.out.println(n);
    n++;
}
System.out.println("after");
```

- a. 10
after
- b. after
- c. It loops for ever
- d. Nothing

X 10
after

Wrong. A while tests first.

✓ after

Correct. $10 < 5$ is false at the start, so the while body never runs.

X It loops for ever

Wrong. The body never runs, so it cannot loop.

X Nothing

Wrong. The line after the loop still runs.

| A while may run zero times; a do-while always runs once.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
for (int i = 10; i > 0; i -= 3) {  
    System.out.print(i + " ");  
}
```

- a. 10 7 4
- b. 9 6 3
- c. 10 7 4 1 -2
- d. 10 7 4 1

X 10 7 4

Wrong. $1 > 0$, so 1 is printed.

X 9 6 3

Wrong. It starts at 10.

X 10 7 4 1 -2

Wrong. -2 fails $i > 0$.

✓ 10 7 4 1

Correct. Start 10, subtract 3 while $i > 0$: 10, 7, 4, 1.

| start; test; step: the test is made before every pass.

Which loop is guaranteed to run its body at least once?

- a. None of them
- b. do-while
- c. while
- d. for

✗ None of them

Wrong. The do-while always runs once.

✓ do-while

Correct. Its test comes after the body.

✗ while

Wrong. A while tests first and may never run.

✗ for

Wrong. A for tests first too.

| for counts, while waits, do-while waits but always goes once. A do-while ends with a semicolon.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int count = 0;
for (int i = 3; i <= 12; i++) {
    count++;
}
System.out.println(count);
```

- a. 10
- b. 9
- c. 11
- d. 12

✓ 10

Correct. With $\leq$, a loop from 3 to 12 runs last minus first plus one times: $12 - 3 + 1 = 10$.

✗ 9

Wrong. $\leq$ includes 12.

✗ 11

Wrong. Count them: 3 to 12 is ten numbers.

✗ 12

Wrong. It starts at 3, not 1.

Count the passes before you trace: last minus first plus one, for a $\leq$ test.

3.16

EASY

Lesson 2.4

Where does the STEP of a for header go in the while version?

- a. Inside the while brackets
- b. On the last line inside the loop body
- c. Above the loop
- d. After the loop

✗ Inside the while brackets

Wrong. That is where the test goes.

✓ On the last line inside the loop body

Correct. Start above, test in the brackets, step at the bottom of the body.

✗ Above the loop

Wrong. That is where the start goes.

✗ After the loop

Wrong. After the loop it would run only once.

Start goes above. Test stays put. Step goes to the bottom of the body.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int total = 0;
for (int i = 2; i <= 8; i += 3) {
    total += i;
}
System.out.println(total);
```

a. 10

b. 20

c. 15

d. 26

X 10

Wrong. 8 is still ≤ 8 , so it is added.

X 20

Wrong. The loop does not reach 11.

✓ 15

Correct. i takes 2, 5 and 8; at 11 the test fails. $2 + 5 + 8 = 15$.

X 26

Wrong. 11 fails the test and is never added.

Write the table: every pass, and the pass that fails the test.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int n = 4071;
int s = 0;
while (n > 0) {
    s += n % 10;
    n /= 10;
}
System.out.println(s);
```

- a. 4071
- b. 12
- c. 11
- d. 4

X 4071

Wrong. The loop sums the digits; it does not rebuild the number.

✓ 12

Correct. The loop pulls off 1, 7, 0 and 4 with % 10 and removes each with / 10: $1 + 7 + 0 + 4 = 12$.

X 11

Wrong. Add every digit: $4 + 0 + 7 + 1$.

X 4

Wrong. That is how many digits there are.

| $n \% 10$ is the last digit and $n /= 10$ removes it: together they walk a number right to left.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int i;  
for (i = 0; i < 4; i++) {  
}  
System.out.println(i);
```

- a. 5
- b. It does not compile
- c. 3
- d. 4

X 5

Wrong. The loop stops as soon as i reaches 4.

X It does not compile

Wrong. i is declared before the loop, so it exists afterwards.

X 3

Wrong. The last step makes i 4 before the failing test.

✓ 4

Correct. i passes 0, 1, 2, 3; then i becomes 4, the test $4 < 4$ fails, and the loop ends with i at 4.

Always write the pass that fails the test: it is where off-by-one answers come from.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int x = 1;
while (x < 50) {
    x *= 3;
}
System.out.println(x);
```

- a. 81
- b. 243
- c. 27
- d. 50

✓ 81

Correct. x goes 1, 3, 9, 27, 81; at 81 the test $x < 50$ fails.

✗ 243

Wrong. The loop stops at 81.

✗ 27

Wrong. 27 is still below 50, so it is tripled again.

✗ 50

Wrong. x only ever holds powers of 3.

The test happens before each pass; the last value is the first one that fails it.

3.21

MEDIUM

Lesson 2.1

Four separate ifs instead of a ladder. What does this print?

```
int score = 95;
char g = 'F';
if (score >= 70) g = 'A';
if (score >= 60) g = 'B';
if (score >= 50) g = 'C';
if (score >= 45) g = 'D';
System.out.println(g);
```

- a. D
- b. F
- c. ABCD
- d. A

✓ D

Correct. All four tests are made; 95 passes each, so each assignment overwrites the last and D is left.

✗ F

Wrong. At least one test matched.

✗ ABCD

Wrong. g holds one char, replaced each time.

✗ A

Wrong. A was set first, then overwritten three times.

Separate ifs are all tested and the last match wins. The paper asks for a ladder, and says so.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
String tier = "Silver";
int days = 2;
int discount;
if (tier.equals("Gold")) {
    if (days == 3) {
        discount = 25;
    } else {
        discount = 15;
    }
} else {
    if (days == 3) {
        discount = 10;
    } else {
        discount = 5;
    }
}
System.out.println("Discount " + discount + "%");
```

- a. Discount 10%
- b. Discount 5%
- c. Discount 15%
- d. Discount 25%

X Discount 10%

Wrong. 10 needs 3 days.

✓ Discount 5%

Correct. Silver is not Gold, so the outer else runs; days is not 3, so the inner else gives 5.

✗ Discount 15%

Wrong. 15 is the Gold path.

✗ Discount 25%

Wrong. 25 is Gold with 3 days.

The outer test picks a pair, the inner test picks one of that pair. Compare Strings with equals.

3.23

MEDIUM

Lesson 2.1

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
int score = 50;
if (0 <= score <= 100) {
    System.out.println("ok");
}
```

- a. It prints true
- b. It does not compile: Java cannot chain comparisons like this
- c. It prints nothing
- d. It prints ok

✗ It prints true

Wrong. It never compiles.

✓ It does not compile: Java cannot chain comparisons like this

Correct. `0 <= score` gives a boolean, and a boolean cannot be compared with `<= 100`.

✗ It prints nothing

Wrong. It never compiles.

✗ It prints ok

Wrong. It never compiles.

Write `score >= 0 && score <= 100`.

3.24

MEDIUM

Lesson 2.1

True or false: without braces, an else attaches to the nearest unmatched if.

Answer: **true**

Braces on every if and every else, always.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int day = 2;
switch (day) {
    case 1:
        System.out.println("one");
    case 2:
        System.out.println("two");
    case 3:
        System.out.println("three");
        break;
    case 4:
        System.out.println("four");
}
```

- a. one
two
three
- b. two
three
- c. two
- d. two
three
four

✗ one
two
three

Wrong. It jumps straight to case 2.

✓ two
three

Correct. It matches case 2, prints, falls into case 3, prints, and meets the break.

✗ two

Wrong. With no break after case 2 it falls into case 3.

✗ two
three
four

Wrong. The break after case 3 stops it.

■ A missing break falls through into the next case until a break or the closing brace.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
double price = 2.5;
switch (price) {
    case 2.5: System.out.println("cheap"); break;
    default: System.out.println("dear");
}
```

- a. It prints nothing
- b. It prints dear
- c. It does not compile: a switch cannot take a double
- d. It prints cheap

✗ It prints nothing

Wrong. It never compiles.

✗ It prints dear

Wrong. It never compiles.

✓ It does not compile: a switch cannot take a double

Correct. Switch on byte, short, int, char, String or an enum; never double or boolean.

✗ It prints cheap

Wrong. It never compiles.

■ You cannot switch on a price, only on something exactly comparable.

Asked twice. This same question also appears as: Lesson 1.5 and Lesson 2.8 and Lesson 3.1 and Lesson 3.5 and Lesson 3.6.

What happens when this runs?

```
int day = 5;
switch (day) {
    case 1: System.out.println(75); break;
    default: throw new IllegalArgumentException("Day must be 1, 2 or 3");
}
```

- a. It does not compile
- b. It prints nothing
- c. It prints 75
- d. It throws `IllegalArgumentException` from the default

✗ It does not compile

Wrong. It compiles.

✗ It prints nothing

Wrong. The throw stops the program with an exception.

✗ It prints 75

Wrong. 75 is only for day 1.

✓ It throws `IllegalArgumentException` from the default

Correct. 5 matches no case, so default throws the exception.

The paper's ticket question rejects an invalid day with throw in the default.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int n = 10;
do {
    System.out.println(n);
    n++;
} while (n < 5);
```

- a. Nothing
- b. 10
- c. It loops for ever
- d. 10
- 11
- 12
- 13
- 14

X Nothing

Wrong. The body runs before the test.

✓ 10

Correct. A do-while runs its body once before testing, and $11 < 5$ is then false.

X It loops for ever

Wrong. The test fails after one pass.

X 10

11

12

13

14

Wrong. After the first pass the test fails.

A do-while always runs at least once: its test comes after the body.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
for (int i = 1; i <= 2; i++) {  
    for (int j = 1; j <= 3; j++) {  
        System.out.print(i * j + " ");  
    }  
    System.out.println();  
}
```

- a. 1 2 3
2 4 6
- b. 1 2 3 2 4 6
- c. 1 2
2 4
3 6
- d. 1
2
3
2
4
6

✓ 1 2 3
2 4 6

Correct. Each outer pass prints a row of three, and println ends the row.

X 1 2 3 2 4 6

Wrong. The println in the outer loop breaks the rows.

X 1 2

2 4

3 6

Wrong. The inner loop runs three times, so each row has three values.

X 1

2

3

2

4

6

Wrong. The inner loop uses print, so a row stays on one line.

Which loop is each statement inside? That decides a row against a grid.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
for (int i = 0; i < 3; i++) {  
}  
System.out.println(i);
```

- a. It does not compile: i exists only inside the for loop
- b. It prints 0
- c. It prints 3
- d. It prints 2

✓ It does not compile: i exists only inside the for loop

Correct. A variable declared in the for header does not exist after the loop.

✗ It prints 0

Wrong. It never compiles.

✗ It prints 3

Wrong. It never compiles.

✗ It prints 2

Wrong. It never compiles.

Declare the counter before the loop if you need it afterwards.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int n = 1;
while (n < 100) {
    n *= 2;
}
System.out.println(n);
```

a. 256

b. 64

c. 100

d. 128

X 256

Wrong. The loop stops at 128.

X 64

Wrong. 64 is still below 100, so it doubles again.

X 100

Wrong. n only ever holds powers of 2.

✓ 128

Correct. n doubles 1, 2, 4, ..., 64, 128; at 128 the test $n < 100$ fails.

As a for loop: `for (int n = 1; n < 100; n *= 2) { }`, and the printed value is the same.

3.32

MEDIUM

Lesson 2.4

True or false: a counter declared in a for header can still be used after the loop.

Answer: **false**

That is the one real difference between the two forms.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 3; j++) {  
        if (j == 3) break;  
        System.out.print(i + " " + j + " ");  
    }  
}
```

- a. 11 21 31
- b. 11 12 21 22 31 32
- c. 11 12 13 21 22 23 31 32 33
- d. 11 12

X 11 21 31

Wrong. The break is at 3, not 2.

✓ 11 12 21 22 31 32

Correct. break leaves only the inner loop, at $j = 3$, so each outer pass prints two values.

X 11 12 13 21 22 23 31 32 33

Wrong. $j = 3$ is never printed.

X 11 12

Wrong. The outer loop carries on after the inner break.

break and continue act on the innermost loop only.

3.34

MEDIUM

Lesson 2.5

The count is placed **ABOVE** the continue. What does this print?

```
int[] a = {5, 12, 3, 20};  
int count = 0;  
for (int i = 0; i < a.length; i++) {  
    count++;  
    if (a[i] < 10) continue;  
}  
System.out.println(count);
```

a. 4

b. 2

c. 0

d. 1

✓ 4

Correct. count++ runs for every item before the skip happens, so it counts all four.

X 2

Wrong. That would need count++ after the continue.

X 0

Wrong. count++ runs every pass.

X 1

Wrong. It runs on all four passes.

Anything above the continue runs for every item, including the ones you meant to skip.

In a FOR loop, what does continue do?

- a. Skips the rest of this pass, runs the step, and goes on to the test
- b. Leaves the loop entirely
- c. Skips the step as well
- d. Restarts the loop from its start value

✓ Skips the rest of this pass, runs the step, and goes on to the test

Correct. In a for loop the step still happens.

✗ Leaves the loop entirely

Wrong. That is break.

✗ Skips the step as well

Wrong. In a for loop the step is not skipped; in a while loop a step at the bottom would be.

✗ Restarts the loop from its start value

Wrong. The counter keeps its value and moves on.

| continue skips one, break stops them all.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q401 {  
    public static void main(String[] args) {  
        int day = 2;  
        switch (day) {  
            case 1: System.out.println("Mon");  
            case 2: System.out.println("Tue");  
            case 3: System.out.println("Wed");  
                break;  
            default: System.out.println("Other");  
        }  
    }  
}
```

- a. Tue
- b. Tue
Wed
- c. Tue
Wed
Other
- d. Mon
Tue
Wed

✗ Tue

Wrong. Without a break at the end of case 2, execution falls straight into case 3.

✓ Tue

Wed

Correct. It enters at case 2, prints Tue, falls through into case 3 and prints Wed, and the break there stops it.

✗ Tue

Wed

Other

Wrong. The break at the end of case 3 stops it before default.

✗ Mon

Tue

Wed

Wrong. It ENTERS at the matching case; the cases above it are skipped entirely.

The answer is Tue then Wed. A switch enters at the matching label and runs on until a break.
Missing breaks are the classic switch bug and a favourite exam question.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q402 {  
    public static void main(String[] args) {  
        int x = 5;  
        if (x > 10)  
            if (x > 20) System.out.println("big");  
            else System.out.println("small");  
        System.out.println("end");  
    }  
}
```

- a. small
end
- b. big
end
- c. end
- d. small

X small
end

Wrong, and it is what the INDENTATION suggests, which is why the question is here. The else does not belong to the outer if.

✗ big
end

Wrong. x is 5, so the outer condition is already false.

✓ end

Correct. An else attaches to the nearest unmatched if, which is the INNER one. Since x is not greater than 10, neither if is entered and nothing but end prints.

✗ small

Wrong. The last println is outside every if and always runs.

The answer is end. An else binds to the nearest if, whatever the indentation says. Braces on every if remove the ambiguity, and they cost nothing.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q403 {  
    public static void main(String[] args) {  
        int sum = 0;  
        for (int i = 1; i <= 5; i++) sum += i;  
        System.out.println(sum);  
        int j = 1, prod = 1;  
        while (j <= 4) { prod *= j; j++; }  
        System.out.println(prod);  
    }  
}
```

- a. 10
24
- b. 15
10
- c. 21
24
- d. 15
24

X 10
24

Wrong. The loop includes 5, because the test is $i \leq 5$, not $i < 5$.

✗ 15

10

Wrong on the second: the while loop multiplies rather than adds.

✗ 21

24

Wrong. 21 would be summing to 6, and the loop stops at 5.

✓ 15

24

Correct. $1+2+3+4+5$ is 15, and $1 \times 2 \times 3 \times 4$ is 24.

The answer is 15 then 24. The three parts of a for header, initialise, test, update, are exactly the three pieces you place by hand when converting to a while. This conversion is worth 10 marks on the 2024/25 paper.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q404 {  
    public static void main(String[] args) {  
        int n = 10;  
        while (n < 5) { System.out.println("while " + n); n++; }  
        do { System.out.println("do " + n); n++; } while (n < 5);  
    }  
}
```

- a. do 10
- b. nothing is printed
- c. while 10
do 10
- d. do 10
do 11

✓ do 10

Correct. The while loop tests first, finds 10 is not less than 5, and never runs. The do-while runs its body once, prints do 10, and only then tests.

✗ nothing is printed

Wrong. The do-while body runs before its condition is ever looked at.

X while 10

do 10

Wrong. The while loop prints nothing at all.

X do 10

do 11

Wrong. After the single pass n is 11, the condition fails, and the loop ends.

The answer is do 10. while tests first and may run zero times; do-while runs at least once. That difference is the whole reason both exist.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q405 {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 5; i++) {  
            if (i == 2) continue;  
            if (i == 4) break;  
            System.out.print(i + " ");  
        }  
    }  
}
```

a. 1 2 3

b. 1 3

c. 1 3 5

d. 1 3 4

✗ 1 2 3

Wrong. The continue skips the print for 2.

✓ 1 3

Correct. 1 prints, 2 is skipped by continue, 3 prints, and at 4 the break leaves the loop before 5 is reached.

✗ 1 3 5

Wrong. The break at 4 stops the loop, so 5 never happens.

X 1 3 4

Wrong. The break happens BEFORE the print, so 4 is never printed.

The answer is 1 3 . continue abandons this pass, break abandons the loop. Read the order of the statements carefully: whether the break is before or after the print changes the answer.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q406 {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= i; j++) System.out.print("*");  
            System.out.println();  
        }  
    }  
}
```

a. ***

b. *

*

*

c. *

**

d. ***

**

*

X ***

Wrong. The inner limit is i, not 3, so it grows with each outer pass.

X *

*

*

Wrong. That would need the inner loop to run exactly once each time.

✓ *

**

Correct. The inner loop runs i times, so one star, then two, then three.

X ***

**

*

Wrong, and it is upside down: i grows, so the rows get longer.

The answer is * then ** then ***. Triangle patterns are a standing favourite. Trace the inner limit for each value of the outer variable and write the row out; three rows of tracing settles any pattern question.

How many times does the body run?

```
public class Q407 {  
    public static void main(String[] args) {  
        int count = 0;  
        for (int i = 0; i < 10; i += 2) count++;  
        System.out.println(count);  
    }  
}
```

- a. 4
- b. 6
- c. 10
- d. 5

✗ 4

Wrong. Count them out: 0, 2, 4, 6, 8. That is five values, not four.

✗ 6

Wrong. 10 is not less than 10, so it never enters with i at 10.

✗ 10

Wrong. The step is 2, so it runs half as often as a step of 1 would.

✓ 5

Correct. i takes 0, 2, 4, 6 and 8, then 10 fails the test $i < 10$, so the body runs five times.

The answer is 5. When a loop steps by more than one, write the values out rather than dividing in your head. The last value is the one people get wrong.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int n = 154, temp = n, sum = 0;
while (temp > 0) {
    int d = temp % 10;
    sum += d * d * d;
    temp /= 10;
}
System.out.println(sum + " " + (sum == n));
```

- a. 190 false
- b. 154 true
- c. 10 false
- d. 190 true

✓ 190 false

Correct. The cubes of 1, 5 and 4 are 1, 125 and 64, which sum to 190, not 154.

✗ 154 true

Wrong. The sum of the cubes is 190.

✗ 10 false

Wrong. The digits are cubed before they are added.

✗ 190 true

Wrong. 190 is not equal to 154.

Trace it digit by digit: pull off the last digit, cube it, add it, remove it.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int x = 3;
if (x > 5); {
    System.out.println("big");
}
System.out.println("end");
```

- a. It does not compile
- b. big
end
- c. end
- d. big

X It does not compile

Wrong. It compiles, which is what makes it dangerous.

✓ big
end

Correct. `if (x > 5);` is a complete, empty if; the braces after it are an ordinary block that always runs.

X end

Wrong. The block is not controlled by the if.

✗ big

Wrong. end prints after the block.

| A semicolon straight after the condition ends the if. The block that follows runs every time.

3.45

HARD

Lesson 2.2

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
int x = 2;
int n = 2;
switch (x) {
    case n: System.out.println("match"); break;
}
```

- a. It prints 2
- b. It prints nothing
- c. It prints match
- d. It does not compile: a case label must be a constant, and n is a variable

✗ It prints 2

Wrong. It never compiles.

✗ It prints nothing

Wrong. It never compiles.

✗ It prints match

Wrong. It never compiles.

✓ It does not compile: a case label must be a constant, and n is a variable

Correct. The compiler builds the jump table before the program runs, so labels must be constants.

| Every case label must be a literal or a final constant.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int count = 0;
for (int i = 0; i < 5; i++);
{
    count++;
}
System.out.println(count);
```

- a. 1
- b. 5
- c. 0
- d. It does not compile

✓ 1

Correct. for (...); is a complete empty loop; the block after it is ordinary code that runs once.

✗ 5

Wrong. The block is not the loop's body.

✗ 0

Wrong. The block runs once.

X It does not compile

Wrong. It compiles, which is what makes it dangerous.

A semicolon after the for line is a complete empty loop.

3.47

HARD

Lesson 2.3

The total is declared **INSIDE** the loop. What does this print?

```
int[] a = {4, 5, 9};
int last = 0;
for (int i = 0; i < a.length; i++) {
    int total = 0;
    total += a[i];
    last = total;
}
System.out.println(last);
```

a. 0

b. It does not compile

c. 9

d. 18

X 0

Wrong. last holds 9.

X It does not compile

Wrong. The program prints last, which is declared outside.

✓ 9

Correct. The last line uses last, which holds the final item. total resets every pass and cannot be seen outside anyway.

X 18

Wrong. Nothing adds all the items outside the loop.

Declaring the total inside the loop resets it every pass; declare it before, add inside, print after.

Which while loop does exactly what this for loop does?

```
int sum = 0;
for (int k = 1; k <= 4; k++) {
    sum += k;
}
System.out.println(sum);
```

- a. `int k = 1; while (k < 4) { sum += k; k++; }`
- b. `int k = 1; while (k <= 4) { sum += k; k++; }`
- c. `int k = 1; while (k <= 4) { k++; sum += k; }`
- d. `int k = 1; while (k <= 4) { sum += k; }`

✗ `int k = 1; while (k < 4) { sum += k; k++; }`

Wrong. The test changed from `<=` to `<`, so 4 is never added.

✓ `int k = 1; while (k <= 4) { sum += k; k++; }`

Correct. Start above, test unchanged, step at the bottom: the sum is 10, the same as the for.

✗ `int k = 1; while (k <= 4) { k++; sum += k; }`

Wrong. Stepping BEFORE adding adds 2, 3, 4 and 5, giving 14.

✗ `int k = 1; while (k <= 4) { sum += k; }`

Wrong. The step is missing, so k stays 1 and the loop never ends.

Start goes above, test stays put, step goes to the bottom. Then check the output is identical.

Half converted: the step is in the header AND the body. What does this print?

```
int total = 0;
for (int i = 1; i <= 10; i++) {
    total += i;
    i++;
}
System.out.println(total);
```

- a. 30
- b. 25
- c. 55
- d. It loops for ever

✗ 30

Wrong. The even numbers are skipped, not the odd.

✓ 25

Correct. i goes up by 2 each pass, 1, 3, 5, 7, 9, so only the odd numbers are added: 25.

✗ 55

Wrong. That is the sum of 1 to 10, which would need one step per pass.

✗ It loops for ever

Wrong. i still passes 10, so it stops.

Leaving the step in the body as well as the header double-steps the counter.

This for loop is converted to a while with `i++` as the last line of the body. What goes wrong?

```
int sum = 0;
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    sum += i;
}
System.out.println(sum);
```

- a. The while version does not compile
- b. Nothing; the while prints the same result
- c. When `i` is 3, `continue` skips the `i++`, so `i` stays 3 and the while loop never ends
- d. The while version skips 4 as well

✗ The while version does not compile

Wrong. It compiles; it just never stops.

✗ Nothing; the while prints the same result

Wrong. `continue` changes the meaning once the step is in the body.

✓ When `i` is 3, `continue` skips the `i++`, so `i` stays 3 and the while loop never ends

Correct. In a `for`, `continue` still runs the step; in the `while`, the step at the bottom is jumped over.

✗ The while version skips 4 as well

Wrong. It does not reach 4 at all.

Convert a loop containing `continue` with care, and say so in your answer.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int[] a = {5, 12, 3, 20, 8, 30};
int count = 0, total = 0;
for (int i = 0; i < a.length; i++) {
    if (a[i] < 10) continue;
    total += a[i];
    count++;
    if (count == 2) break;
}
System.out.println(total + " " + count);
```

- a. 62 3
- b. 12 1
- c. 78 6
- d. 32 2

X 62 3

Wrong. The break at a count of 2 stops before 30.

X 12 1

Wrong. 20 is added before the break.

X 78 6

Wrong. Items under 10 are skipped, and the loop breaks early.

✓ 32 2

Correct. 5 and 3 are skipped; 12 and 20 are added and counted; at a count of 2 the loop breaks before 30.

■ Skip first, then process, then count, then decide whether to stop: the Section A pattern.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
double t = 1.1 * 3;  
System.out.println(t);  
System.out.printf("%.1f%n", t);
```

a. 3.3000000000000003

3.30

b. 3.3

3.30

c. 3.3

3.3

d. 3.3000000000000003

3.3

X 3.3000000000000003

3.30

Wrong. %.1f shows ONE decimal place.

X 3.3

3.30

Wrong. The first line shows the tail.

✗ 3.3

3.3

Wrong. println shows the exact double, tail included.

✓ 3.3000000000000003

3.3

Correct. A double cannot hold 1.1 exactly, so the raw value shows a tail; printf with %.1f shows one decimal place.

Compute money in double, print it with printf and a stated number of decimal places.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
boolean stop = false;
int found = -1;
for (int i = 1; i <= 5 && !stop; i++) {
    for (int j = 1; j <= 5; j++) {
        if (i * j == 6) {
            found = i * 10 + j;
            stop = true;
            break;
        }
    }
}
System.out.println(found);
```

- a. 16
- b. -1
- c. 23
- d. 32

X 16

Wrong. found is set as $i * 10 + j$.

X -1

Wrong. A pair is found.

✓ 23

Correct. The first pair with $i * j == 6$ is $i = 2, j = 3$; the flag then stops the outer loop too.

✗ 32

Wrong. $i = 2, j = 3$ is found before $i = 3, j = 2$.

! To leave two loops, set a boolean flag and test it in the outer condition.

4 Methods

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static int sq(int n) {  
    return n * n;  
}  
  
// in main:  
System.out.println(sq(3) + sq(4));
```

- a. 12
- b. 25
- c. 14
- d. 7

X 12

Wrong. $9 + 16$ is 25.

✓ 25

Correct. $\text{sq}(3)$ returns 9 and $\text{sq}(4)$ returns 16; $9 + 16$ is 25.

X 14

Wrong. Squaring, not doubling.

X 7

Wrong. sq squares; it does not return its argument.

| A call to a value returning method is a value, so two can be added.

4.2

EASY

Lesson 2.6

A method is declared `area(double w, double h)` and called as `area(width, 3.0)`. Which are the arguments?

- a. width and 3.0
- b. area and double
- c. w and h
- d. w and 3.0

✓ width and 3.0

Correct. Arguments are the values supplied at the call.

X area and double

Wrong. Those are the name and a type.

X w and h

Wrong. w and h are the parameters, the names in the header.

X w and 3.0

Wrong. w is a parameter.

| The parameter is the name in the header; the argument is the value passed at the call.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q501 {  
    static void change(int n) { n = 99; }  
    public static void main(String[] args) {  
        int x = 5;  
        change(x);  
        System.out.println(x);  
    }  
}
```

- a. 5
- b. 99
- c. 0
- d. it does not compile

✓ 5

Correct. Java passes arguments BY VALUE. The parameter `n` is a separate variable holding a copy of 5, so assigning to it cannot reach `x`.

✗ 99

Wrong. The method changed its own copy of the value, not the caller's variable.

✗ 0

Wrong. `x` was assigned 5 and nothing since has touched it.

X it does not compile

Wrong. The code is perfectly legal; it simply does not do what it looks like it does.

The answer is 5. Java is pass by value, always. For an object the VALUE passed is the reference, which is why an object's contents can be changed by a method while the caller's variable still points at the same object.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q502 {  
    static void fill(int[] a) { a[0] = 99; }  
    static void replace(int[] a) { a = new int[]{7, 7}; }  
    public static void main(String[] args) {  
        int[] nums = {1, 2};  
        fill(nums);  
        replace(nums);  
        System.out.println(nums[0] + " " + nums[1]);  
    }  
}
```

- a. 1 2
- b. 99 2
- c. 7 7
- d. 99 7

X 1 2

Wrong. fill really did change the array, because both names refer to the same array object.

✓ 99 2

Correct. fill reaches through the reference and changes element 0. replace only rebinds its own copy of the reference, so the caller's array is untouched.

X 7 7

Wrong. replace cannot change which array the caller's variable points at.

X 99 7

Wrong for the same reason: nothing from replace escapes the method.

The answer is 99 2. Through the reference you can change the object; assigning to the parameter only changes where that copy points. This one pair of methods is the clearest statement of what pass by value means in Java.

4.5

MEDIUM

Manual O.5 Q4

Why can a static method not use an instance field directly?

- a. Because static methods are private
- b. Because instance fields are always final
- c. Because static methods run before the class is loaded
- d. Because a static method may run with no object in existence, so there is no instance whose field it could mean

X Because static methods are private

Wrong. static and private are unrelated: a static method can be public.

X Because instance fields are always final

Wrong. Instance fields are ordinarily variable; final is a separate choice.

X Because static methods run before the class is loaded

Wrong. The class is loaded before any of its methods run.

✓ Because a static method may run with no object in existence, so there is no instance whose field it could mean

Correct. Static belongs to the class, so it can be called with no object at all. An instance field only exists inside an object, so the reference would be to nothing.

The answer is Because a static method may run with no object in existence, so there is no instance whose field it could mean. Static belongs to the class, instance belongs to the object. This is exactly why main is static: the JVM calls it before any object of your class exists.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
static void show(int n) {  
    System.out.println("the number is " + n);  
}
```

```
// in main:  
int y = show(5);
```

- a. It does not compile: show is void, so there is no value to store
- b. It prints 5
- c. It prints the number is 5, then 0
- d. It prints the number is 5

✓ It does not compile: show is void, so there is no value to store

Correct. A void call is a statement, not a value.

✗ It prints 5

Wrong. It never compiles.

✗ It prints the number is 5, then 0

Wrong. It never compiles.

✗ It prints the number is 5

Wrong. It never compiles.

A void method performs an action and returns nothing; a value returning method hands a result back with return.

twice is not static. What happens when this is compiled?

```
int twice(int n) {  
    return 2 * n;  
}  
  
// in main:  
System.out.println(twice(4));
```

- a. It prints 8
- b. It does not compile: main is static and there is no object to call twice on
- c. It prints 4
- d. It prints 0

✗ It prints 8

Wrong. It never compiles.

✓ It does not compile: main is static and there is no object to call twice on

Correct. A method called from main without an object must be static.

✗ It prints 4

Wrong. It never compiles.

✗ It prints 0

Wrong. It never compiles.

Forgetting static on a helper is one of the commonest first-exam compile errors.

What exactly is a method signature?

- a. The method's first line, including public static
- b. The method name together with the number, types and order of its parameters
- c. The return type and the method name
- d. The values passed at the call

✗ The method's first line, including public static

Wrong. Modifiers are not part of it.

✓ The method name together with the number, types and order of its parameters

Correct. And it does NOT include the return type.

✗ The return type and the method name

Wrong. The return type is not part of the signature.

✗ The values passed at the call

Wrong. Those are arguments.

Signature is name plus parameter list. Never the return type.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
class Box {
    static int made = 0;
    int id;
    Box() {
        made++;
        id = made;
    }
}

// in main:
Box a = new Box();
Box b = new Box();
Box c = new Box();
System.out.println(b.id + " " + Box.made);
```

- a. 2 3
- b. 3 3
- c. 1 3
- d. 2 1

✓ 2 3

Correct. made is one shared counter, 3 after three objects; b was the second, so its id is 2.

✗ 3 3

Wrong. Each object has its own id.

✗ 1 3

Wrong. b was made second.

✗ 2 1

Wrong. made is shared, so it counts all three.

static is one shared copy; an instance field is one copy per object.

4.10

MEDIUM

Lesson 2.7

count is an instance field. What happens when this is compiled?

```
int count = 0;

// in main:
count++;
System.out.println(count);
```

- a. It prints 1
- b. It does not compile: a static method cannot use an instance field directly
- c. It prints 0
- d. It prints nothing

✗ It prints 1

Wrong. It never compiles.

✓ It does not compile: a static method cannot use an instance field directly

Correct. main is static and may run when no object exists, so there is no count to mean.

✗ It prints 0

Wrong. It never compiles.

✗ It prints nothing

Wrong. It never compiles.

"non-static variable cannot be referenced from a static context" is the commonest first-exam compile error.

4.11

MEDIUM

Lesson 2.7

Do `void f(int a, double b)` and `void f(double a, int b)` overload legally in one class?

- a. Only if one is static
- b. Yes, because the order of the parameter types differs
- c. No, because both return void
- d. No, because they have the same types

✗ Only if one is static

Wrong. static has nothing to do with it.

✓ Yes, because the order of the parameter types differs

Correct. Same types in a different order makes a different signature.

✗ No, because both return void

Wrong. The return type plays no part.

✗ No, because they have the same types

Wrong. Order is part of the signature.

Number, types and order: change any of the three and the signature differs.

4.12

MEDIUM

Lesson 2.7

True or false: an overloaded call is resolved at run time, from the actual object.

Answer: **false**

Overloading is static, or compile time, polymorphism.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q503 {  
    static String f(int n) { return "int"; }  
    static String f(double d) { return "double"; }  
    static String f(String s) { return "String"; }  
    public static void main(String[] args) {  
        System.out.println(f(1));  
        System.out.println(f(1.0));  
        System.out.println(f("1"));  
    }  
}
```

- a. double
double
String
- b. int
int
String
- c. int
double
String
- d. it does not compile, because three methods share a name

✗ double
double
String

Wrong. 1 is an int literal and an exact int match exists, so no widening is needed.

✗ int
int
String

Wrong. 1.0 is a double literal, so the double version matches exactly.

✓ int
double
String

Correct. Overload resolution picks by the argument TYPES: 1 is an int, 1.0 is a double, "1" is a String.

✗ it does not compile, because three methods share a name

Wrong. Sharing a name with different parameter lists is exactly what overloading IS.

The answer is int then double then String. Overloading is same name, different parameter list, resolved at compile time by the argument types. Return type alone cannot distinguish two overloads.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q505 {
    static int addUp(int n) {
        int total = 0;
        for (int i = 1; i <= n; i++) {
            total += i;
            if (total > 6) return total;
        }
        return -1;
    }
    public static void main(String[] args) {
        System.out.println(addUp(5));
    }
}
```

- a. 10
- b. 15
- c. -1
- d. 6

✓ 10

Correct. The totals are 1, 3, 6, 10. The first that is greater than 6 is 10, and return hands it back immediately.

✗ 15

Wrong. 15 is the full sum to 5, but the method leaves as soon as the running total passes 6.

X -1

Wrong. The final return is only reached if the loop finishes without the condition ever being true.

X 6

Wrong. 6 is not GREATER than 6, so the loop continues one more pass.

The answer is 10. return leaves the method at once, loop and all. Note the strictness of the comparison: greater than 6 excludes 6 itself.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static void fill(int[] a) {  
    a[0] = 42;  
}  
  
// in main:  
int[] v = {1, 2};  
fill(v);  
System.out.println(v[0] + " " + v.length);
```

- a. 1 2
- b. It does not compile
- c. 42 1
- d. 42 2

X 1 2

Wrong. A change made through the copied reference reaches the same array.

X It does not compile

Wrong. It compiles.

X 42 1

Wrong. The array still has two elements.

✓ 42 2

Correct. The method gets a copy of the REFERENCE, which points at the same array, so the change is visible.

Java is always pass by value; for an object the value copied is the reference.

4.16 **HARD** Lesson 2.6

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
static int sign(int n) {  
    if (n > 0) {  
        return 1;  
    } else if (n < 0) {  
        return -1;  
    }  
}  
  
// in main:  
System.out.println(sign(5));
```

- a. It does not compile: there is a path ($n == 0$) with no return
- b. It prints -1
- c. It prints 0
- d. It prints 1

✓ It does not compile: there is a path ($n == 0$) with no return

Correct. A value returning method must return on every route out.

✗ It prints -1

Wrong. It never compiles.

✗ It prints 0

Wrong. Java does not invent a return value.

✗ It prints 1

Wrong. It never compiles.

Every route out of a value returning method must return, including the one nobody tested.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static int f(int n) {  
    System.out.print("f" + n + " ");  
    return n + 1;  
}  
  
// in main:  
int r = f(f(1));  
System.out.println(r);
```

- a. f2 f1 3
- b. f1 f2 3
- c. f1 f2 2
- d. f1 3

X f2 f1 3

Wrong. The inner call happens first.

✓ f1 f2 3

Correct. The inner call f(1) runs first, printing f1 and returning 2; then f(2) prints f2 and returns 3.

X f1 f2 2

Wrong. f(2) returns 3.

X f1 3

Wrong. f is called twice.

Arguments are worked out before the method they are passed to runs.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static String kind(int x) { return "int"; }
static String kind(double x) { return "double"; }
static String kind(String x) { return "String"; }

// in main:
System.out.println(kind(7) + " " + kind(7.5) + " " + kind('A'));
```

- a. int double String
- b. It does not compile
- c. int double char
- d. int double int

X int double String

Wrong. 'A' is a char, not a String.

X It does not compile

Wrong. Each call matches a method.

X int double char

Wrong. There is no kind(char), so 'A' widens to int.

✓ int double int

Correct. 7 matches int, 7.5 matches double, and 'A' has no char version, so it widens to int.

The compiler chooses by the signature: the number, types and order of the arguments.

4.19

HARD

Lesson 2.7

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
static int f(int a) { return a; }  
static double f(int a) { return a; }
```

```
// in main:  
System.out.println(f(2));
```

- a. It does not compile: the two methods have the same signature
- b. It prints 2
- c. It prints 2.0
- d. It prints 2 and 2.0

✓ It does not compile: the two methods have the same signature

Correct. The signature is the name plus the parameter list; the return type is not part of it.

✗ It prints 2

Wrong. It never compiles.

✗ It prints 2.0

Wrong. It never compiles.

✗ It prints 2 and 2.0

Wrong. It never compiles.

Two methods differing only in return type cannot coexist: the compiler could never tell them apart.

5 Object-Oriented Programming

5.1

EASY

Manual O.6 Q2

Which pair of choices implements encapsulation?

- a. Private fields with public getters and setters
- b. Public fields and no methods
- c. Static fields with a public constructor
- d. Final fields with no constructor

✓ Private fields with public getters and setters

Correct. The data is hidden and every route to it goes through a method, which is where validation can live.

✗ Public fields and no methods

Wrong, and it is the opposite: public fields let anything change the object into an invalid state with no check possible.

✗ Static fields with a public constructor

Wrong. static concerns whether the field belongs to the class or the object, and has nothing to do with hiding it.

✗ Final fields with no constructor

Wrong. final stops reassignment; it does not hide anything, and a class with no constructor still gets the default one.

The answer is Private fields with public getters and setters. Private data, public methods. Hiding the data is only half of it: the point of the getter and setter is that they are a place to check.

Which statement is TRUE of an abstract class?

- a. It can be instantiated directly with new
- b. All of its methods must be abstract
- c. A class may extend several abstract classes at once
- d. It may contain both abstract methods and fully implemented methods

✗ It can be instantiated directly with new

Wrong. An abstract class is incomplete by definition, so new on it is a compile error. Its subclasses are what get instantiated.

✗ All of its methods must be abstract

Wrong. It may have none at all and still be marked abstract.

✗ A class may extend several abstract classes at once

Wrong. Java allows single inheritance of classes. Several INTERFACES may be implemented, which is how the restriction is worked around.

✓ It may contain both abstract methods and fully implemented methods

Correct. That is the difference from an interface in the classic form: an abstract class can carry shared state and shared behaviour alongside the methods it leaves to subclasses.

The answer is It may contain both abstract methods and fully implemented methods. Abstract class for shared state and partial implementation; interface for a contract that many unrelated classes can sign. One superclass, many interfaces.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
class Counter {
    int n;
    void inc() {
        n++;
    }
}

// in main:
Counter a = new Counter();
Counter b = new Counter();
a.inc();
a.inc();
b.inc();
System.out.println(a.n + " " + b.n);
```

- a. 2 1
- b. 1 1
- c. 3 0
- d. 3 3

✓ 2 1

Correct. Each object has its own n: a was incremented twice and b once.

✗ 1 1

Wrong. a.inc() ran twice.

✗ 3 0

Wrong. b.inc() ran once.

✗ 3 3

Wrong. n is an instance field, one per object.

■ An attribute is one copy per object; that is the difference between a class and its objects.

5.4

EASY

Lesson 3.1

What two things make a constructor different from every other method?

- a. It has the same name as the class, and no return type at all
- b. It is static and returns void
- c. It is private and takes no parameters
- d. It is called with a dot on an object

✓ It has the same name as the class, and no return type at all

Correct. Those two together are how Java recognises it.

✗ It is static and returns void

Wrong. A constructor has no return type, not even void.

✗ It is private and takes no parameters

Wrong. Constructors are usually public and often take parameters.

✗ It is called with a dot on an object

Wrong. It is run by new.

■ A constructor shares the class name and has no return type; new runs it.

Which statement correctly defines an OBJECT?

- a. A variable declared final
- b. A blueprint that defines attributes and methods
- c. An instance of a class, created with new, with its own copy of the attributes
- d. A method with no return type

X A variable declared final

Wrong. That is a constant.

X A blueprint that defines attributes and methods

Wrong. That is a class.

✓ An instance of a class, created with new, with its own copy of the attributes

Correct. That is an object.

X A method with no return type

Wrong. That describes a constructor.

| Class is the blueprint, object is the building, new is the builder.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
class Student {  
    private int score;  
}  
  
// in main:  
Student st = new Student();  
st.score = 50;  
System.out.println(st.score);
```

- a. It throws an exception
- b. It does not compile: score is private to Student
- c. It prints 50
- d. It prints 0

X It throws an exception

Wrong. It is refused at compile time.

✓ It does not compile: score is private to Student

Correct. A private field is reachable only from inside its own class. Go through a setter.

X It prints 50

Wrong. It never compiles.

X It prints 0

Wrong. It never compiles.

| Private data, public doors.

Which two are benefits of encapsulation?

- a. The program always runs faster
- b. Validation: a setter can refuse an invalid value
- c. Control: only the class's own methods change its data
- d. A class can extend two parents

✗ The program always runs faster

Wrong. Speed is not a benefit claimed.

✓ Validation: a setter can refuse an invalid value

Correct. Validation is one benefit.

✓ Control: only the class's own methods change its data

Correct. Control is one benefit.

✗ A class can extend two parents

Wrong. Java has no multiple inheritance of classes.

Control, validation and safe change: the internal representation can change without breaking callers.

How many classes can one Java class extend?

- a. None; Java has no inheritance
- b. Two
- c. Exactly one
- d. As many as it likes

✗ None; Java has no inheritance

Wrong. extends is inheritance.

✗ Two

Wrong. One only.

✓ Exactly one

Correct. Java has no multiple inheritance of classes; a class may implement many interfaces instead.

✗ As many as it likes

Wrong. That is true of interfaces, not classes.

One parent only; use interfaces for more.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
class Animal {
    void breathe() {
        System.out.println("breathing");
    }
}
class Fish extends Animal {
    void swim() {
        System.out.println("swimming");
    }
}

// in main:
Fish f = new Fish();
f.breathe();
f.swim();
```

- a. breathing
- b. It does not compile
- c. breathing
swimming
- d. swimming

X breathing

Wrong. swim() is Fish's own method.

✗ It does not compile

Wrong. It compiles.

✓ breathing
swimming

Correct. Fish inherits breathe() from Animal and adds swim().

✗ swimming

Wrong. breathe() is inherited, so it runs too.

! The child gets the parent's public and protected members for free, and adds what is new.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q606 {
    static class Point {
        int x = 1, y = 2;
        @Override public String toString() { return "(" + x + ", " + y + ")"; }
    }
    public static void main(String[] args) {
        System.out.println(new Point());
    }
}
```

- a. (1, 2)
- b. Point@1b6d3586
- c. Point
- d. it does not compile

✓ (1, 2)

Correct. println on an object calls its toString(), and this class overrides it.

✗ Point@1b6d3586

Wrong for THIS class. That class-name-and-hash form is what the INHERITED toString gives, and it is what you see when you forget to override it.

✗ Point

Wrong. Nothing prints just the class name.

X it does not compile

Wrong. Overriding toString is ordinary and expected.

The answer is (1, 2). println on an object calls toString(). Override it and your objects become printable; forget to, and you get the class name and a hash.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
class Item {  
    String name;  
    double price;  
    boolean sold;  
}  
  
// in main:  
Item i = new Item();  
System.out.println(i.name + " " + i.price + " " + i.sold);
```

- a. null 0 null
- b. "" 0.0 false
- c. null 0.0 false
- d. 0 0 false

X null 0 null

Wrong. price is a double, 0.0, and sold is a boolean, false.

X "" 0.0 false

Wrong. An unset String is null, not empty text.

✓ null 0.0 false

Correct. An unset object reference is null, a double is 0.0, a boolean is false.

✗ 0 0 false

Wrong. A String field starts as null, not 0.

Defaults for fields never set: numbers 0, boolean false, every object reference null.

5.12

MEDIUM

Lesson 3.1

Asked twice. This same question also appears as: Lesson 1.5 and Lesson 2.2 and Lesson 2.8 and Lesson 3.5 and Lesson 3.6.

What happens when this runs?

```
class Item {  
    String name;  
}  
  
// in main:  
Item i = new Item();  
System.out.println(i.name.length());
```

- a. It prints null
- b. It throws NullPointerException, because name was never set
- c. It does not compile
- d. It prints 0

✗ It prints null

Wrong. The length() call fails before anything prints.

✓ It throws NullPointerException, because name was never set

Correct. The field is null, and calling a method on null has no object to ask.

✗ It does not compile

Wrong. It compiles; the fault shows only when it runs.

✗ It prints 0

Wrong. There is no String whose length could be 0.

That is the argument for setting every field in the constructor.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
class Student {
    private int score;
    public void setScore(int s) {
        if (s >= 0 && s <= 100) {
            score = s;
        }
    }
    public int getScore() {
        return score;
    }
}

// in main:
Student st = new Student();
st.setScore(80);
st.setScore(150);
System.out.println(st.getScore());
```

- a. 150
- b. 0
- c. It does not compile
- d. 80

X 150

Wrong. The setter refuses anything above 100.

✗ 0

Wrong. The first call set it to 80.

✗ It does not compile

Wrong. It compiles.

✓ 80

Correct. 80 is accepted; 150 is refused by the check, so the score stays 80.

Encapsulation is what makes validation possible: the setter checks before it changes anything.

5.14

MEDIUM

Lesson 3.2

Which list runs from **MOST open** to **MOST closed**?

- a. private, default, protected, public
- b. public, default, protected, private
- c. protected, public, private, default
- d. public, protected, default, private

✗ private, default, protected, public

Wrong. That is most closed first.

✗ public, default, protected, private

Wrong. protected is more open than default.

✗ protected, public, private, default

Wrong. public is the most open.

✓ public, protected, default, private

Correct. Anywhere; subclasses and package; package only; the class itself only.

Fields private, methods public: private data, public behaviour.

From where can a protected member be reached?

- a. The class itself only
- b. Other packages only
- c. Anywhere at all
- d. The class itself, its subclasses, and the same package

✗ The class itself only

Wrong. That is private.

✗ Other packages only

Wrong. Not what protected means.

✗ Anywhere at all

Wrong. That is public.

✓ The class itself, its subclasses, and the same package

Correct. Mine, my children's, and my package's.

| protected earns its place when a subclass needs a parent's field.

5.16

MEDIUM

Lesson 3.2

What is the difference between encapsulation and abstraction?

- a. They are the same idea
- b. Encapsulation hides the data; abstraction hides the implementation
- c. Encapsulation hides the implementation; abstraction hides the data
- d. Abstraction is about constructors

✗ They are the same idea

Wrong. They are separate pillars.

✓ Encapsulation hides the data; abstraction hides the implementation

Correct. Two different pillars, two different answers.

✗ Encapsulation hides the implementation; abstraction hides the data

Wrong. This reverses them.

✗ Abstraction is about constructors

Wrong. Not what abstraction means.

Encapsulation: data behind methods. Abstraction: what a thing does, not how.

5.17

MEDIUM

Lesson 3.2

True or false: writing getters and setters while leaving the fields public is still encapsulation.

Answer: **false**

Private data, public doors. A setter with no check buys nothing either.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
class Account {
    double balance;
    Account(double b) {
        balance = b;
        System.out.println("Account " + b);
    }
}
class Savings extends Account {
    double rate;
    Savings(double b, double r) {
        super(b);
        rate = r;
        System.out.println("Savings " + r);
    }
}

// in main:
Savings s = new Savings(100, 0.1);
System.out.println(s.balance * s.rate);
```

a. Account 100.0

Savings 0.1

10.0

b. Account 100.0

10.0

c. Savings 0.1

Account 100.0

10.0

d. 10.0

✓ Account 100.0

Savings 0.1

10.0

Correct. super(b) runs the parent constructor first, then the rest of the child's; balance times rate is 10.0.

✗ Account 100.0

10.0

Wrong. The child's constructor prints its line too.

✗ Savings 0.1

Account 100.0

10.0

Wrong. The parent's constructor runs first, through super.

✗ 10.0

Wrong. Both constructors print.

super(...) calls the parent's constructor and must be the first statement in the child's.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
class Account {
    double balance;
    Account(double b) {
        balance = b;
    }
}
class Savings extends Account {
    Savings(double b) {
        System.out.println("x");
        super(b);
    }
}

// in main:
Savings s = new Savings(5);
```

- a. It prints nothing
- b. It prints x and 5.0
- c. It does not compile: super(...) must be the first statement
- d. It prints x

✗ It prints nothing

Wrong. It never compiles.

✗ It prints x and 5.0

Wrong. It never compiles.

✓ It does not compile: super(...) must be the first statement

Correct. The parent's part must exist before the child adds to it.

✗ It prints x

Wrong. It never compiles.

extends takes it, super builds it, and super goes first.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
class Account {  
    private double balance;  
}  
class Savings extends Account {  
    void add() {  
        balance += 1;  
    }  
}  
  
// in main:  
new Savings().add();
```

- a. It does not compile: balance is private, so Savings cannot see it
- b. It throws an exception
- c. It prints 1.0
- d. It prints 0.0

✓ It does not compile: balance is private, so Savings cannot see it
Correct. Use protected, or go through a getter.

✗ It throws an exception
Wrong. It is a compile error.

✗ It prints 1.0
Wrong. It never compiles.

✗ It prints 0.0
Wrong. It never compiles.

private is invisible even to a child; protected means mine and my children's.

Should Car extend Engine?

- a. Yes: a car is an engine
- b. No: a car HAS an engine, so Engine should be a field of Car
- c. No: Java cannot model cars
- d. Yes: extends is the only way to reuse code

✗ Yes: a car is an engine

Wrong. A car is not an engine.

✓ No: a car HAS an engine, so Engine should be a field of Car

Correct. Inheritance is an is a; a field is a has a.

✗ No: Java cannot model cars

Wrong. Not the reason.

✗ Yes: extends is the only way to reuse code

Wrong. A field reuses the engine's code too.

| If you cannot say "a child IS A parent" with a straight face, you want a field.

Which two are NOT inherited by a subclass?

- a. Constructors
- b. Public methods
- c. Private members
- d. Protected fields

✓ Constructors

Correct. A constructor belongs to its own class; it is called with super.

✗ Public methods

Wrong. Public methods are inherited.

✓ Private members

Correct. Private members exist in the child object but the child's code cannot see them.

✗ Protected fields

Wrong. Protected fields are inherited.

Inherited: public and protected. Not inherited: constructors and anything private.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
class A {  
    void show() { }  
}  
class B extends A {  
    @Override  
    void shwo() { }  
}  
  
// in main:  
new B().show();
```

- a. It compiles, and shwo is a new method
- b. It prints nothing
- c. It does not compile: shwo does not override anything, and @Override makes that an error
- d. It throws an exception

X It compiles, and shwo is a new method

Wrong. Without @Override that would happen silently; with it, it is an error.

X It prints nothing

Wrong. It never compiles.

✓ It does not compile: shwo does not override anything, and @Override makes that an error

Correct. @Override asks the compiler to check, so the typo is caught.

X It throws an exception

Wrong. It is a compile error.

| Always write @Override: a misspelt name becomes an error instead of a silent new method.

Which three facts distinguish overloading from overriding?

- a. Same class against subclass; parameters differ against identical; compile time against run time
- b. Loop against recursion; array against String; input against output
- c. Public against private; static against final; int against double
- d. There is no difference

✓ Same class against subclass; parameters differ against identical; compile time against run time

Correct. Class, parameters, timing.

✗ Loop against recursion; array against String; input against output

Wrong. Unrelated.

✗ Public against private; static against final; int against double

Wrong. Not the differences the paper wants.

✗ There is no difference

Wrong. They differ in three independent ways.

Class, parameters, timing: three words, three marks.

5.25

MEDIUM

Lesson 3.4

An Account variable holds a Savings object, and Savings overrides describe(). Whose describe() runs?

- a. Both, one after the other
- b. Savings's, decided at run time from the actual object
- c. Account's, decided at compile time from the declared type
- d. Neither; it does not compile

✗ Both, one after the other

Wrong. One version runs.

✓ Savings's, decided at run time from the actual object

Correct. The actual object decides, at run time.

✗ Account's, decided at compile time from the declared type

Wrong. That is how overloading is decided, not overriding.

✗ Neither; it does not compile

Wrong. A parent-typed variable may hold a child.

Polymorphism: one call, many forms, chosen at run time by the object.

5.26

MEDIUM

Lesson 3.4

True or false: overriding is decided at compile time from the declared type.

Answer: **false**

Overloading at compile time, overriding at run time.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
class Emp {
    double pay() {
        return 100;
    }
}
class Manager extends Emp {
    double pay() {
        return 250;
    }
}

// in main:
Emp[] staff = {new Emp(), new Manager(), new Manager()};
double total = 0;
for (int i = 0; i < staff.length; i++) {
    total += staff[i].pay();
}
System.out.println(total);
```

- a. 350.0
- b. 600.0
- c. 600
- d. 300.0

✗ 350.0

Wrong. There are two Managers.

✓ 600.0

Correct. Each object's own `pay()` runs: $100 + 250 + 250 = 600.0$.

✗ 600

Wrong. total is a double, so it prints 600.0.

✗ 300.0

Wrong. The Manager objects run Manager's `pay()`.

■ The loop never has to know the kinds: each object answers with its own version.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q601 {
    static class Dog {
        String name;
        Dog() { this("unnamed"); }
        Dog(String n) { name = n; }
    }
    public static void main(String[] args) {
        System.out.println(new Dog().name);
        System.out.println(new Dog("Rex").name);
    }
}
```

- a. null
Rex
- b. unnamed
unnamed
- c. it does not compile
- d. unnamed
Rex

X null
Rex

Wrong. The no-argument constructor does not leave name unset: it calls the other constructor with a default.

X unnamed
unnamed

Wrong. The second call passes Rex explicitly, and that is what is stored.

X it does not compile

Wrong. Constructor chaining with `this(...)` is legal, provided it is the first statement.

✓ unnamed
Rex

Correct. `this("unnamed")` chains to the other constructor, so the first object gets unnamed and the second gets Rex.

The answer is unnamed then Rex. A constructor has the class name, no return type, and runs on new. `this(...)` chains to another constructor and must be the first statement in the body.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q603 {
    static class Animal {
        void speak() { System.out.println("some sound"); }
    }
    static class Dog extends Animal {
        @Override void speak() { System.out.println("woof"); }
    }
    public static void main(String[] args) {
        Animal a = new Dog();
        a.speak();
    }
}
```

- a. some sound
- b. woof
- c. it does not compile
- d. both lines print, superclass first

X some sound

Wrong. The DECLARED type is Animal, but the method that runs is chosen by the object's ACTUAL type at run time.

✓ woof

Correct. The object is a Dog, so Dog's override runs. This is polymorphism, and it is exactly what makes inheritance useful.

X it does not compile

Wrong. Assigning a subclass object to a superclass variable is always allowed: a Dog IS an Animal.

X both lines print, superclass first

Wrong. An override replaces the superclass version. Running both would need an explicit `super.speak()` call.

The answer is woof. The variable's type decides what you may CALL; the object's type decides what actually RUNS. That sentence answers most polymorphism questions on this paper.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q604 {  
    static class A {  
        A() { System.out.println("A"); }  
    }  
    static class B extends A {  
        B() { System.out.println("B"); }  
    }  
    public static void main(String[] args) { new B(); }  
}
```

- a. B
- b. B
A
- c. A
B
- d. nothing prints

X B

Wrong. The superclass constructor always runs first, whether or not you wrote super().

X B

A

Wrong, and it is the wrong way round: the base is built before the part built on top of it.

✓ A

B

Correct. Java inserts an implicit `super()` at the top of B's constructor, so A's constructor completes before B's body starts.

✗ nothing prints

Wrong. `new B()` runs both constructors.

The answer is A then B. Constructors run from the top of the hierarchy down. A subclass object cannot exist until the superclass part of it has been built.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
class Pet {  
    String name;  
    Pet(String name) {  
        name = name;  
    }  
}  
  
// in main:  
Pet p = new Pet("Rex");  
System.out.println(p.name);
```

- a. Rex
- b. It does not compile
- c. null
- d. name

X Rex

Wrong. Without this, the field is never assigned.

X It does not compile

Wrong. It compiles, which is why it is dangerous.

✓ null

Correct. `name = name` assigns the parameter to itself; the field is never set and stays null.

✗ name

Wrong. `name` is a variable, not the text "name".

| `this.name = name`, on every line of every constructor.

5.32

HARD

Lesson 3.1

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
class Student {  
    String name;  
    Student(String name) {  
        this.name = name;  
    }  
}  
  
// in main:  
Student s = new Student();  
System.out.println(s.name);
```

- a. It prints nothing
- b. It prints 0
- c. It prints null
- d. It does not compile: writing your own constructor removes the free no-argument one

✗ It prints nothing

Wrong. It never compiles.

✗ It prints 0

Wrong. It never compiles.

✗ It prints null

Wrong. It never compiles.

✓ It does not compile: writing your own constructor removes the free no-argument one

Correct. Once you write any constructor, `new Student()` no longer exists unless you write it too.

| Constructors can be overloaded; write both if you need both.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
class Pet {
    String name = "unknown";
    void Pet(String n) {
        name = n;
    }
}

// in main:
Pet p = new Pet();
System.out.println(p.name);
```

- a. unknown
- b. null
- c. It does not compile
- d. Rex

✓ unknown

Correct. `void Pet(String n)` is an ordinary method, not a constructor, so `new Pet()` uses the default and the field keeps unknown.

✗ null

Wrong. `name` was given the starting value `unknown`.

X It does not compile

Wrong. A method may share the class name; it compiles.

X Rex

Wrong. Nothing ever calls the void method.

| Giving a constructor a return type turns it into an ordinary method.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
class Account {
    double balance;
    Account(double b) {
        balance = b;
    }
}
class Savings extends Account {
    Savings() {
        System.out.println("new");
    }
}

// in main:
Savings s = new Savings();
```

- a. It prints new and 0.0
- b. It does not compile: Savings must call super(...) because Account has no no-argument constructor
- c. It prints new
- d. It throws an exception

X It prints new and 0.0

Wrong. It never compiles.

✓ It does not compile: Savings must call super(...) because Account has no no-argument constructor

Correct. Java inserts a call to super(), which does not exist here.

X It prints new

Wrong. It never compiles.

X It throws an exception

Wrong. It is a compile error.

"constructor Account in class Account cannot be applied to given types" means one thing: you forgot super.

The child's describe takes a parameter. What does this print?

```
class Account {  
    void describe() {  
        System.out.println("account");  
    }  
}  
class Savings extends Account {  
    void describe(String s) {  
        System.out.println("savings " + s);  
    }  
}  
  
// in main:  
Account a = new Savings();  
a.describe();
```

- a. account
- b. It does not compile
- c. savings null
- d. savings

✓ account

Correct. describe(String) has different parameters, so it OVERLOADS; nothing overrides describe(), and the parent's version runs.

✗ It does not compile

Wrong. It compiles.

✗ savings null

Wrong. The call passes no argument, so describe(String) is not used.

✗ savings

Wrong. Nothing overrides the no-argument describe().

Changing the parameters while meaning to override overloads instead. @Override would have caught it.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
class A {  
    public void show() { }  
}  
class B extends A {  
    private void show() { }  
}  
  
// in main:  
new A().show();
```

- a. It prints nothing
- b. It does not compile: an override may not be more restrictive than the method it replaces
- c. It compiles, and B.show is private
- d. It throws an exception

✗ It prints nothing

Wrong. It never compiles.

✓ It does not compile: an override may not be more restrictive than the method it replaces

Correct. A public method cannot be overridden as private.

✗ It compiles, and B.show is private

Wrong. Narrowing the access is refused.

✗ It throws an exception

Wrong. It is a compile error.

The fourth difference: an overridden method may not be made less accessible.

6 Strings

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
String a = "Gold";  
String b = "gold";  
System.out.println(a.equals(b) + " " + a.equalsIgnoreCase(b));
```

- a. false false
- b. false true
- c. true true
- d. true false

X false false

Wrong. equalsIgnoreCase ignores case.

✓ false true

Correct. equals compares characters exactly, so case matters; equalsIgnoreCase does not care about case.

X true true

Wrong. Gold and gold differ in case.

X true false

Wrong. They are the other way round.

Always compare text with equals; use equalsIgnoreCase for anything the user typed.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= 3; i++) {
    sb.append(i).append("-");
}
System.out.println(sb.toString());
```

- a. -1-2-3
- b. 1-2-3-
- c. 1-2-3
- d. 123

X -1-2-3

Wrong. The number comes before its dash.

✓ 1-2-3-

Correct. Each pass appends the number and a dash, including after the last.

X 1-2-3

Wrong. A dash is appended after 3 too.

X 123

Wrong. Dashes are appended.

6.3

EASY

Lesson 2.9

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
System.out.println("banana".replace("a", "o"));
```

- a. bnn
- b. banana
- c. bonono
- d. bonana

X bnn

Wrong. The a's become o's; they are not removed.

X banana

Wrong. The replaced copy is what is printed.

✓ bonono

Correct. replace swaps EVERY a for an o and returns the new String.

X bonana

Wrong. Every occurrence is replaced, not just the first.

| replace(a, b) returns a copy with every a swapped for b.

The paper says to build a list "using plain String concatenation only, no StringBuilder".
What do you use?

- a. Whichever you prefer
- b. StringBuilder, because it is more efficient
- c. Plain concatenation with +, because the paper said so
- d. A char array

✗ Whichever you prefer

Wrong. The constraint decides.

✗ StringBuilder, because it is more efficient

Wrong. It is better in general, but the paper forbade it here.

✓ Plain concatenation with +, because the paper said so

Correct. Where the paper bans a tool, using it costs the marks whatever the output.

✗ A char array

Wrong. Not what the paper asked for.

Read the constraint before you choose the tool.

Why is `StringBuilder` preferred to `String` for building text in a loop?

- a. It is shorter to type
- b. `String` is immutable, so each join creates a new object, while `StringBuilder` changes one buffer in place
- c. `StringBuilder` can hold more characters than `String`
- d. `String` cannot be used inside a loop

✗ It is shorter to type

Wrong. It is longer to type, which is why the reason has to be a real one.

✓ `String` is immutable, so each join creates a new object, while `StringBuilder` changes one buffer in place

Correct. Joining strings in a loop makes a fresh object every pass and throws the previous one away. `StringBuilder` appends to a single mutable buffer.

✗ `StringBuilder` can hold more characters than `String`

Wrong. Neither has a practical limit you will meet on this course.

✗ `String` cannot be used inside a loop

Wrong. It can be used, it is just wasteful.

The answer is `String` is immutable, so each join creates a new object, while `StringBuilder` changes one buffer in place. Immutable means every join allocates. In a loop of n joins that is n objects created and discarded, which is why `StringBuilder` exists.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
String s = "java";
String t = s.toUpperCase();
s.concat("!");
System.out.println(s + " " + t);
```

- a. JAVA! JAVA
- b. java JAVA
- c. java! JAVA
- d. JAVA JAVA

X JAVA! JAVA

Wrong. s is still java.

✓ java JAVA

Correct. Every String method returns a new String; s is never changed, and the concat result was thrown away.

X java! JAVA

Wrong. The concat result was never assigned.

X JAVA JAVA

Wrong. toUpperCase made a copy for t; s is unchanged.

A String is immutable: if you did not assign the result, nothing happened.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
String s = "  hi  ";  
System.out.println "[" + s.trim() + "]" + s.length();
```

- a. [hi]6
- b. [hi]2
- c. [hi]6
- d. [hi]4

X [hi]6

Wrong. trim() removes the spaces in the copy that is printed.

X [hi]2

Wrong. s.length() measures the untrimmed s.

✓ [hi]6

Correct. trim() returns a copy without the spaces; s itself still has all six characters.

X [hi]4

Wrong. Two spaces on each side plus two letters is six.

| trim() returns a trimmed copy; the original keeps its spaces.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
String s = "hello";  
System.out.println(s.length);
```

- a. It does not compile: a String's length is a method, length()
- b. It prints 4
- c. It prints 0
- d. It prints 5

✓ It does not compile: a String's length is a method, length()

Correct. Strings have a length() method; arrays have a length field.

✗ It prints 4

Wrong. It never compiles.

✗ It prints 0

Wrong. It never compiles.

✗ It prints 5

Wrong. It never compiles.

| s.length() for a String, a.length for an array.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
System.out.println("banana".indexOf("x") + " " + "banana".indexOf("an"));
```

a. -1 1

b. -1 2

c. -1 3

d. 0 1

✓ -1 1

Correct. x is absent, so -1; the first an starts at index 1.

✗ -1 2

Wrong. b is index 0, a is index 1.

✗ -1 3

Wrong. indexOf finds the FIRST occurrence.

✗ 0 1

Wrong. indexOf gives -1 when the text is absent.

indexOf(x) is where x starts, or -1 if it is not there.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q701 {  
    public static void main(String[] args) {  
        String a = "hello";  
        String b = "hello";  
        String c = new String("hello");  
        System.out.println(a == b);  
        System.out.println(a == c);  
        System.out.println(a.equals(c));  
    }  
}
```

- a. true
true
true
- b. false
false
true
- c. true
false
true
- d. true
false

false

X true
true
true

Wrong on the second line. `new String()` deliberately builds a separate object, so the two references differ.

X false
false
true

Wrong on the first line: string literals are interned, so both names point at the same pooled object.

✓ true
false
true

Correct. Two identical literals share one pooled object, so `a == b` is true. `new String` makes a distinct object, so `a == c` is false. `equals` compares the CHARACTERS, so it is true.

X true
false
false

Wrong on the last line. `equals` compares contents, and the contents are the same.

The answer is true then false then true. `==` asks whether they are the same OBJECT; `equals` asks whether they read the same. For strings, always use `equals`. This is the most asked string question there is.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q702 {  
    public static void main(String[] args) {  
        String s = "java";  
        s.toUpperCase();  
        System.out.println(s);  
        System.out.println(s.toUpperCase());  
    }  
}
```

a. JAVA
JAVA

b. java
java

c. JAVA
java

d. java
JAVA

X JAVA
JAVA

Wrong on the first line. The result of the first call was thrown away, because nothing was assigned.

✗ java
java

Wrong on the second line: the value returned by the call is what is printed, and it is uppercase.

✗ JAVA
java

Wrong, and it has the two lines the wrong way round.

✓ java
JAVA

Correct. Strings are immutable, so `toUpperCase` returns a NEW string and leaves `s` alone. The first call changes nothing; the second prints its result directly.

The answer is java then JAVA. A String method never changes the string. If you do not catch what it returns, nothing happened. A statement that is only a method call on a String is almost always a bug.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q703 {  
    public static void main(String[] args) {  
        String s = "PROGRAM";  
        System.out.println(s.length());  
        System.out.println(s.charAt(0));  
        System.out.println(s.substring(1, 4));  
        System.out.println(s.indexOf("G"));  
    }  
}
```

a. 7

P

ROG

3

b. 7

P

ROGR

3

c. 7

R

ROG

4

d. 6
P
ROG
3

✓ 7
P
ROG
3

Correct. Seven characters; index 0 is P; substring(1, 4) takes indexes 1, 2 and 3, stopping BEFORE 4; and the first G is at index 3.

✗ 7
P
ROGR
3

Wrong. substring stops before its second index, so it takes three characters, not four.

✗ 7
R
ROG
4

Wrong. charAt(0) is the first character, P. Java indexes from zero.

✗ 6
P
ROG
3

Wrong. PROGRAM has seven letters.

The answer is 7 then P then ROG then 3. length() has brackets on a String and none on an array, index from zero, and substring excludes its end index. Those three answer most string questions.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
String s = "EXAMINATION";  
System.out.println(s.substring(2, 5) + " " + s.charAt(4) + " " + s.indexOf("NATION"));
```

- a. AMI I 5
- b. AMIN I 5
- c. AMI N 5
- d. AMI I 6

✓ AMI I 5

Correct. substring(2, 5) takes indexes 2 to 4; charAt(4) is I; NATION starts at index 5.

✗ AMIN I 5

Wrong. substring stops BEFORE its end index.

✗ AMI N 5

Wrong. Index 4 is I; N is at 5.

✗ AMI I 6

Wrong. Count from 0: N of NATION is at index 5.

| substring(a, b) runs from a up to but not including b; indexes start at 0.

7 Arrays

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int[] a = {4, 8, 15, 16};  
System.out.println(a[a.length - 1] + " " + a.length);
```

- a. 16 3
- b. 16 4
- c. an `ArrayIndexOutOfBoundsException`
- d. 15 4

✗ 16 3

Wrong. length counts the elements, 4.

✓ 16 4

Correct. The last index is `length - 1`, which is 3, holding 16; length is 4.

✗ an `ArrayIndexOutOfBoundsException`

Wrong. `a.length - 1` is a valid index.

✗ 15 4

Wrong. Index 3 holds 16.

■ The last index is always `length - 1`.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
int[] a = {1.5, 2.0};  
System.out.println(a[0]);
```

- a. It prints 2
- b. It does not compile: 1.5 and 2.0 are doubles, which do not fit an int array
- c. It prints 1
- d. It prints 1.5

✗ It prints 2

Wrong. It never compiles.

✓ It does not compile: 1.5 and 2.0 are doubles, which do not fit an int array

Correct. This is the planted error from the last paper: fix it with double[] or whole values.

✗ It prints 1

Wrong. It never compiles.

✗ It prints 1.5

Wrong. It never compiles.

| An int[] filled with decimals does not compile, and it is on the 2025/2026 paper.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q802 {  
    public static void main(String[] args) {  
        int[] a = {1, 2, 3};  
        try {  
            System.out.println(a[3]);  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("caught " + e.getMessage());  
        }  
    }  
}
```

- a. 3
- b. caught null
- c. caught Index 3 out of bounds for length 3
- d. 0

X 3

Wrong. Index 3 is the FOURTH element, and there are only three.

X caught null

Wrong. The exception carries a message describing the index and the length.

✓ caught Index 3 out of bounds for length 3

Correct. Valid indexes are 0, 1 and 2, so `a[3]` throws, and the catch prints the exception's own message.

✗ 0

Wrong. Reading past the end does not return a default; it throws.

The answer is caught Index 3 out of bounds for length 3. An array of length `n` has indexes 0 to `n` minus 1. The last valid index is always length minus one, and this off-by-one is the most common runtime error in the whole course.

7.4

MEDIUM

Lesson 2.8

Asked twice. This same question also appears as: Lesson 1.5 and Lesson 2.2 and Lesson 3.1 and Lesson 3.5 and Lesson 3.6.

What happens when this runs?

```
int[] a = {4, 8, 15, 16};  
System.out.println(a[a.length]);
```

- a. It prints 0
- b. It prints 16
- c. It throws `ArrayIndexOutOfBoundsException`
- d. It does not compile

✗ It prints 0

Wrong. Java does not give a default for a missing index.

✗ It prints 16

Wrong. 16 is at index 3.

✓ It throws `ArrayIndexOutOfBoundsException`

Correct. Valid indexes run 0 to 3; index 4 is past the end.

✗ It does not compile

Wrong. It compiles; the fault shows only when it runs.

`i <= a.length` is one step too far, every time.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static int countAbove(int[] a, int mark) {
    int n = 0;
    for (int i = 0; i < a.length; i++) {
        if (a[i] > mark) {
            n++;
        }
    }
    return n;
}

// in main:
System.out.println(countAbove(new int[]{3, 9, 12, 9}, 9));
```

- a. 0
- b. 1
- c. 2
- d. 3

X 0

Wrong. 12 is above 9.

✓ 1

Correct. Only 12 is strictly above 9; the two 9s are not.

X 2

Wrong. Only one value is above 9.

X 3

Wrong. > does not include 9.

Count matching: $n = 0$, if the test passes $n++$, return n .

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static int firstIndexOf(int[] a, int wanted) {
    for (int i = 0; i < a.length; i++) {
        if (a[i] == wanted) {
            return i;
        }
    }
    return -1;
}

// in main:
int[] a = {7, 3, 7, 1};
System.out.println(firstIndexOf(a, 7) + " " + firstIndexOf(a, 5));
```

- a. 0 -1
- b. 0 0
- c. 1 -1
- d. 2 -1

✓ 0 -1

Correct. 7 is found first at index 0 and the method returns at once; 5 is absent, so -1.

✗ 0 0

Wrong. -1 means not found.

✗ 1 -1

Wrong. Index 0 holds 7.

✗ 2 -1

Wrong. The method returns the FIRST match.

Return from inside the loop on the first match; return -1 after the loop.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int[][] m = {{1, 2}, {3, 4, 5}};  
System.out.println(m[1].length + " " + m[1][2]);
```

- a. 2 3
- b. 2 5
- c. 3 5
- d. 3 4

X 2 3

Wrong. m.length would be 2; m[1].length is 3.

X 2 5

Wrong. m[1].length is the width of row 1, which is 3.

✓ 3 5

Correct. Row 1 is {3, 4, 5}: its length is 3 and its index 2 is 5.

X 3 4

Wrong. m[1][2] is the third item of row 1.

| The first index picks the row, the second picks within it; each row has its own length.

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
int[] a = {1, 2, 3};  
System.out.println(a.length());
```

- a. It does not compile: an array's length is a field, written without brackets
- b. It prints 3
- c. It prints 0
- d. It prints 2

✓ It does not compile: an array's length is a field, written without brackets

Correct. Arrays have a length field; Strings have a length() method.

✗ It prints 3

Wrong. It never compiles.

✗ It prints 0

Wrong. It never compiles.

✗ It prints 2

Wrong. It never compiles.

| a.length for an array, s.length() for a String. The examiner plants the swap.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
int[] a = {2, 3, 4, 4};
int s = 0;
for (int i = 0; i < a.length; i++) {
    s += a[i];
}
System.out.println((double) s / a.length);
```

- a. 13
- b. 3.0
- c. 3
- d. 3.25

X 13

Wrong. That is the total, not the average.

X 3.0

Wrong. The cast happens before the division, so the remainder survives.

X 3

Wrong. The cast makes this a double division.

✓ 3.25

Correct. The total is 13, and casting it to double before dividing by 4 gives 3.25.

| An average of an int array needs a cast on one side of the division.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q801 {  
    public static void main(String[] args) {  
        int[] a = new int[3];  
        a[0] = 5;  
        System.out.println(a.length);  
        System.out.println(a[1]);  
        System.out.println(a[0]);  
    }  
}
```

- a. 3
null
5
- b. 3
0
5
- c. 2
0
5
- d. 3
0
0

✗ 3
null
5

Wrong. null is the default for an OBJECT array. A numeric array defaults to zero.

✓ 3
0
5

Correct. length is a FIELD on an array, with no brackets. An int array is filled with 0 when created, and a[0] was then set to 5.

✗ 2
0
5

Wrong. new int[3] gives three elements, indexed 0, 1 and 2.

✗ 3
0
0

Wrong. a[0] was assigned 5 on the second line.

The answer is 3 then 0 then 5. a.length with no brackets for an array, s.length() with brackets for a String. Arrays are created filled with the zero of their type: 0, 0.0, false or null.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q803 {  
    public static void main(String[] args) {  
        int[][] m = {{1, 2, 3}, {4, 5, 6}};  
        System.out.println(m.length);  
        System.out.println(m[0].length);  
        System.out.println(m[1][2]);  
    }  
}
```

a. 3

2

6

b. 2

3

5

c. 6

3

6

d. 2

3

6

X 3

2

6

Wrong, and it swaps rows and columns. The outer length is always the number of rows.

X 2

3

5

Wrong. `m[1][2]` is the third element of the second row, which is 6, not 5.

X 6

3

6

Wrong. `m.length` counts rows, not the total number of elements.

✓ 2

3

6

Correct. `m.length` is the number of ROWS, two. `m[0].length` is the length of the first row, three. `m[1][2]` is row 1, column 2, which is 6.

The answer is 2 then 3 then 6. Outer index is the row, inner index is the column, and both count from zero. `m.length` is rows; `m[i].length` is the width of row `i`.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q804 {  
    public static void main(String[] args) {  
        int[] a = {-5, -2, -9};  
        int max = 0;  
        for (int x : a) if (x > max) max = x;  
        System.out.println(max);  
    }  
}
```

- a. 0
- b. -2
- c. -9
- d. -5

✓ 0

Correct, and it is the bug. Starting max at 0 means no negative value can ever beat it, so 0 survives even though it is not in the array.

✗ -2

Wrong, and it is the answer the program was MEANT to give. It does not, because of the way max was initialised.

✗ -9

Wrong. -9 is the smallest, not the largest.

X -5

Wrong. -2 is larger than -5, so -5 could not win even with a correct start.

The answer is 0. Start max at the FIRST ELEMENT, never at zero. This exact fault is a standing exam favourite, and the fix is one line: `int max = a[0]`, then loop from index 1.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static int countConsecutive(int[] a) {
    int n = 0;
    for (int i = 1; i < a.length; i++) {
        if (a[i] == a[i - 1]) {
            n++;
        }
    }
    return n;
}

// in main:
System.out.println(countConsecutive(new int[]{1, 1, 1, 2, 2, 3}));
```

- a. 5
- b. 3
- c. 2
- d. 4

X 5

Wrong. Each match compares a position with the one before it.

✓ 3

Correct. Indexes 1 and 2 equal the 1 before them, and index 4 equals the 2 before it.

X 2

Wrong. A run of three 1s gives two matches, not one.

X 4

Wrong. The 3 at the end has no equal neighbour.

█ The neighbour walk starts at 1 and compares $a[i]$ with $a[i - 1]$; a run of three gives two matches.

8 Recursion

8.1

EASY

Manual O.9 Q3

Which statement about recursion compared with iteration is TRUE?

- a. Recursion is always faster because it uses fewer instructions
- b. Any recursive method is impossible to write as a loop
- c. Recursion uses stack space for every pending call, so a deep recursion can exhaust it
- d. Recursion never needs a stopping condition

✗ Recursion is always faster because it uses fewer instructions

Wrong. Recursion is usually SLOWER: every call costs a stack frame and the work of calling.

✗ Any recursive method is impossible to write as a loop

Wrong. Anything recursive can be written iteratively, sometimes with an explicit stack of your own.

✓ Recursion uses stack space for every pending call, so a deep recursion can exhaust it

Correct. Each call in progress keeps its frame until it returns, so depth costs memory, and running out of it is `StackOverflowError`.

✗ Recursion never needs a stopping condition

Wrong. Without a base case it never stops, and the program dies.

The answer is Recursion uses stack space for every pending call, so a deep recursion can exhaust it. Recursion buys clarity on naturally recursive problems and pays for it in stack space and call overhead. Both sides of that trade earn marks.

What is the BASE CASE of a recursive method?

- a. The condition under which it returns without calling itself
- b. The call that passes a smaller input
- c. The first call made from main
- d. The largest value it can handle

✓ The condition under which it returns without calling itself

Correct. Every recursive method needs one, or it never stops.

✗ The call that passes a smaller input

Wrong. That is the recursive case.

✗ The first call made from main

Wrong. That is just the first call.

✗ The largest value it can handle

Wrong. Not a definition in the course.

Base case stops it, recursive case shrinks it.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static int pow(int b, int e) {  
    if (e == 0) {  
        return 1;  
    }  
    return b * pow(b, e - 1);  
}  
  
// in main:  
System.out.println(pow(2, 5));
```

- a. 64
- b. 32
- c. 16
- d. 10

X 64

Wrong. That is 2 to the power 6.

✓ 32

Correct. $\text{pow}(2, 5)$ is $2 \times 2 \times 2 \times 2 \times 2 \times 1 = 32$.

X 16

Wrong. That is 2 to the power 4.

X 10

Wrong. That multiplies 2 by 5.

Base case $e == 0$ returns 1; each call multiplies by b and shrinks e .

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static void down(int n) {  
    if (n < 1) {  
        return;  
    }  
    System.out.print(n + " ");  
    down(n - 1);  
}  
  
// in main:  
down(3);
```

- a. 3
- b. 3 2 1 0
- c. 1 2 3
- d. 3 2 1

X 3

Wrong. down calls itself until n is below 1.

X 3 2 1 0

Wrong. At 0 the base case returns without printing.

X 1 2 3

Wrong. That is the order when the print comes after the call.

✓ 3 2 1

Correct. Each call prints before it recurses, so the numbers come out on the way down.

Where the print sits relative to the recursive call decides the order.

8.5

MEDIUM

Lesson 3.5

Asked twice. This same question also appears as: Lesson 1.5 and Lesson 2.2 and Lesson 2.8 and Lesson 3.1 and Lesson 3.6.

What happens when this runs?

```
static int bad(int n) {  
    return bad(n - 1);  
}  
  
// in main:  
System.out.println(bad(3));
```

- a. It fails with `StackOverflowError`
- b. It loops quietly for ever
- c. It prints 0
- d. It does not compile

✓ It fails with `StackOverflowError`

Correct. With no base case every call makes another, until the call stack is full.

✗ It loops quietly for ever

Wrong. Each call uses a stack frame, so it runs out of memory quickly.

✗ It prints 0

Wrong. Nothing ever returns.

✗ It does not compile

Wrong. It compiles.

A recursion with no base case is the recursive infinite loop: `StackOverflowError`.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static int ds(int n) {  
    if (n < 10) {  
        return n;  
    }  
    return n % 10 + ds(n / 10);  
}  
  
// in main:  
System.out.println(ds(9045));
```

- a. 18
- b. 9045
- c. 4
- d. 27

✓ 18

Correct. $5 + 4 + 0 + 9 = 18$: each call peels the last digit off with `% 10`.

✗ 9045

Wrong. The method adds the digits.

✗ 4

Wrong. That counts the digits.

X 27

Wrong. Add each digit once.

| The base case is a single digit; the recursive case adds the last digit to the sum of the rest.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static String reverse(String s) {  
    if (s.length() <= 1) {  
        return s;  
    }  
    return reverse(s.substring(1)) + s.charAt(0);  
}  
  
// in main:  
System.out.println(reverse("CODE"));
```

- a. CODE
- b. EDOC
- c. DEOC
- d. ODEC

X CODE

Wrong. The method reverses the text.

✓ EDOC

Correct. Each call puts the first character on the END of the reversed rest.

X DEOC

Wrong. Trace one level at a time.

✗ ODEC

Wrong. Trace it: `reverse("ODE") + 'C'`, and so on down.

| `reverse(s.substring(1)) + s.charAt(0)`: first character last.

8.8

MEDIUM

Lesson 3.5

Which uses more memory, and why?

- a. Iteration, because a loop keeps every value
- b. Recursion uses no memory at all
- c. They use exactly the same
- d. Recursion, because every call in progress holds a stack frame until the base case returns

✗ Iteration, because a loop keeps every value

Wrong. A loop reuses the same variables.

✗ Recursion uses no memory at all

Wrong. Each call occupies memory.

✗ They use exactly the same

Wrong. Each recursive call adds a frame.

✓ Recursion, because every call in progress holds a stack frame until the base case returns

Correct. A loop uses one set of variables however many times it goes round.

| Recursion is clearer for self-similar problems and heavier in memory; iteration is the default.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q901 {
    static int fact(int n) {
        if (n <= 1) return 1;
        return n * fact(n - 1);
    }
    public static void main(String[] args) {
        System.out.println(fact(5));
    }
}
```

- a. 120
- b. 15
- c. 5
- d. StackOverflowError

✓ 120

Correct. $5 \times 4 \times 3 \times 2 \times 1$ is 120, and the base case at $n \leq 1$ stops the recursion.

✗ 15

Wrong. 15 is the SUM $1+2+3+4+5$. This method multiplies.

✗ 5

Wrong. The method does not simply return its argument.

X StackOverflowError

Wrong. There IS a base case and each call moves towards it, so the recursion terminates.

The answer is 120. Every recursion needs a base case and a step that moves towards it. Remove either and you get StackOverflowError, which is what the paper asks you to identify.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q902 {
    static int fib(int n) {
        if (n <= 1) return n;
        return fib(n - 1) + fib(n - 2);
    }
    public static void main(String[] args) {
        for (int i = 0; i < 7; i++) System.out.print(fib(i) + " ");
    }
}
```

- a. 1 1 2 3 5 8 13
- b. 0 1 1 2 3 5 8
- c. 0 1 2 3 4 5 6
- d. 1 2 3 5 8 13 21

X 1 1 2 3 5 8 13

Wrong. This version starts at 0, because fib(0) returns 0.

✓ 0 1 1 2 3 5 8

Correct. The base case returns n itself for 0 and 1, so the sequence starts 0, 1, and each later term is the sum of the two before it.

X 0 1 2 3 4 5 6

Wrong. That is just the counting numbers, not a sum of the two previous terms.

X 1 2 3 5 8 13 21

Wrong. Seven terms from index 0 ends at 8, not 21.

The answer is 0 1 1 2 3 5 8 . Two base cases here, folded into one line: $\text{fib}(0)$ is 0 and $\text{fib}(1)$ is 1.
Trace the first four calls on paper and the pattern becomes obvious.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static void up(int n) {  
    if (n < 1) {  
        return;  
    }  
    up(n - 1);  
    System.out.print(n + " ");  
}  
  
// in main:  
up(3);
```

- a. 3
- b. 0 1 2 3
- c. 1 2 3
- d. 3 2 1

X 3

Wrong. Every level prints once.

X 0 1 2 3

Wrong. At 0 nothing is printed.

✓ 1 2 3

Correct. Each call recurses FIRST and prints afterwards, so the prints happen on the way back up.

✗ 3 2 1

Wrong. That would need the print before the call.

Nothing is done on the way down that comes after the call; it all happens on the way back up.

9 Exceptions and File Input and Output

9.1

EASY

Manual O.10 Q3

Asked twice. This same question also appears as: Manual O.2 Q5.

Which statement is TRUE?

- a. A checked exception must be caught or declared with throws
- b. An unchecked exception must always be caught
- c. All exceptions in Java are checked
- d. A checked exception cannot be caught

✓ A checked exception must be caught or declared with throws

Correct. The compiler insists: handle it, or declare that your method may throw it. `IOException` is the one this course meets most.

✗ An unchecked exception must always be caught

Wrong. Unchecked exceptions, the `RuntimeException` family, need neither. `NullPointerException` is the standard example.

✗ All exceptions in Java are checked

Wrong. Everything descending from `RuntimeException` and `Error` is unchecked.

✗ A checked exception cannot be caught

Wrong. It can be caught; it simply may not be ignored.

The answer is A checked exception must be caught or declared with throws. Checked means the compiler makes you deal with it. Unchecked means it is your job to avoid it, and the commonest ones, null pointer and array index, are bugs rather than conditions.

Which class does this course use to read a text file line by line, and what must surround it?

- a. System.out, inside a for loop
- b. Scanner or BufferedReader, inside a try block that handles IOException
- c. FileWriter, inside a switch
- d. String, with no exception handling needed

✗ System.out, inside a for loop

Wrong. System.out is the output stream, not a way in.

✓ Scanner or BufferedReader, inside a try block that handles IOException

Correct. Either class can read lines, and file work throws a CHECKED exception, so it must be caught or declared.

✗ FileWriter, inside a switch

Wrong. FileWriter writes. Reading needs a reader.

✗ String, with no exception handling needed

Wrong. String is a type, not a file, and the exception handling is not optional for file access.

The answer is Scanner or BufferedReader, inside a try block that handles IOException. Reading needs a reader and reading can fail, so it needs a catch. Both halves of that sentence earn marks.

Which goes in a method HEADER to say the method may raise an exception it does not handle?

- a. finally
- b. catch
- c. throw
- d. throws

✗ finally

Wrong. finally is the cleanup block.

✗ catch

Wrong. catch handles one.

✗ throw

Wrong. throw raises one now, inside the body.

✓ throws

Correct. throws, with the s, declares it.

throw raises one, throws warns about one.

Asked twice. This same question also appears as: Lesson 3.7 and Lesson 3.7 and Lesson 3.7.

main declares throws IOException. What does this print?

```
PrintWriter out = new PrintWriter("log.txt");
out.println("first");
out.close();
PrintWriter more = new PrintWriter(new FileWriter("log.txt", true));
more.println("second");
more.close();
Scanner in = new Scanner(new File("log.txt"));
while (in.hasNextLine()) {
    System.out.println(in.nextLine());
}
in.close();
```

- a. second
first
- b. second
- c. first
- d. first
second

X second
first

Wrong. Appended lines go at the end.

X second

Wrong. The true keeps what was there.

X first

Wrong. The appended line is there too.

✓ first
second

Correct. FileWriter with true opens for append, so the new line goes after the old one.

| new PrintWriter(new FileWriter(name, true)): the true is the whole difference.

Asked twice. This same question also appears as: Lesson 3.7 and Lesson 3.7 and Lesson 3.7.

main declares throws IOException. What does this print?

```
PrintWriter out = new PrintWriter("r.txt");
out.println("a");
out.println("b");
out.println("c");
out.close();
Scanner in = new Scanner(new File("r.txt"));
int n = 0;
while (in.hasNextLine()) {
    in.nextLine();
    n++;
}
in.close();
System.out.println(n);
```

- a. 0
- b. 2
- c. 4
- d. 3

X 0

Wrong. The writer was closed, so the lines reached the file.

X 2

Wrong. All three lines are read.

X 4

Wrong. Only three lines were written.

✓ 3

Correct. hasNextLine keeps the loop going for each of the three lines written.

| while (in.hasNextLine()) reads a file of any length.

Which line opens records.txt to ADD records without destroying the existing ones?

- a. `new FileWriter("records.txt", false)`
- b. `new PrintWriter("records.txt")`
- c. `new PrintWriter(new FileWriter("records.txt", true))`
- d. `new Scanner(new File("records.txt"))`

✗ `new FileWriter("records.txt", false)`

Wrong. false means overwrite.

✗ `new PrintWriter("records.txt")`

Wrong. That wipes the file.

✓ `new PrintWriter(new FileWriter("records.txt", true))`

Correct. FileWriter with true appends.

✗ `new Scanner(new File("records.txt"))`

Wrong. A Scanner reads; it does not write.

The file question always asks you to add to the file you made earlier. That one true is the mark.

Why do the file listings write throws IOException on main?

- a. To make the program run faster
- b. File operations raise checked exceptions, which must be caught or declared
- c. Because every Java program needs it
- d. To close the file automatically

✗ To make the program run faster

Wrong. It has nothing to do with speed.

✓ File operations raise checked exceptions, which must be caught or declared

Correct. Declaring them is the short, legal way to satisfy the compiler.

✗ Because every Java program needs it

Wrong. Only methods that can raise checked exceptions need it.

✗ To close the file automatically

Wrong. You must still close the file.

✓ Catch them, or declare throws IOException.

Asked twice. This same question also appears as: Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4 and Lesson 3.6.

What happens when this is compiled?

```
public class Q1002 {  
    public static void main(String[] args) {  
        try {  
            int x = 1 / 0;  
        } catch (Exception e) {  
            System.out.println("general");  
        } catch (ArithmeticException e) {  
            System.out.println("arithmetic");  
        }  
    }  
}
```

- a. It prints general
- b. It prints arithmetic
- c. It prints both lines
- d. It does not compile: the second catch can never be reached

X It prints general

Wrong. It never runs at all, because it does not compile.

X It prints arithmetic

Wrong for the same reason, and the general catch would win in any case.

X It prints both lines

Wrong. At most one catch clause ever runs for a given exception.

✓ It does not compile: the second catch can never be reached

Correct. Exception is the superclass of ArithmeticException, so the first catch already covers everything the second could. Java rejects unreachable catch clauses at compile time.

The answer is It does not compile: the second catch can never be reached. Catch the most specific exception FIRST and the general one last. Reversing them is a compile error, not a subtle bug, which is the helpful part.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q1005 {  
    public static void main(String[] args) {  
        String s = null;  
        try {  
            System.out.println(s.length());  
        } catch (NullPointerException e) {  
            System.out.println("null pointer");  
        }  
    }  
}
```

- a. 0
- b. null
- c. null pointer
- d. it does not compile

X 0

Wrong. There is no object to measure, so nothing returns 0.

X null

Wrong. Printing s alone would show null; calling a METHOD on it throws.

✓ null pointer

Correct. `s` refers to no object, so calling a method on it throws `NullPointerException`, which the catch handles.

X it does not compile

Wrong. It compiles perfectly. The fault only appears when it runs, which is what unchecked means.

The answer is null pointer. null means the reference points at no object. Reading it is fine; calling a method on it is not. Check for null before you dereference.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
try {  
    int[] a = new int[2];  
    a[2] = 5;  
    System.out.println("stored");  
} catch (ArithmeticException e) {  
    System.out.println("math");  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("index");  
} finally {  
    System.out.println("done");  
}
```

- a. stored
done
- b. index
done
- c. math
done
- d. index

X stored
done

Wrong. The failing line stops the try before stored is printed.

✓ index
done

Correct. `a[2]` is past the end, so the `ArrayIndexOutOfBoundsException` catch runs, then finally.

✗ math
done

Wrong. Nothing divides by zero.

✗ index

Wrong. finally always runs.

✓ The matching catch runs; finally runs every time.

Asked twice. This same question also appears as: Lesson 1.5 and Lesson 2.2 and Lesson 2.8 and Lesson 3.1 and Lesson 3.5.

What happens when this runs?

```
try {  
    String s = null;  
    System.out.println(s.length());  
} catch (ArithmeticException e) {  
    System.out.println("math");  
}
```

- a. A `NullPointerException` stops the program; the `ArithmeticException` catch does not match it
- b. It prints math
- c. It does not compile
- d. It prints 0

✓ A `NullPointerException` stops the program; the `ArithmeticException` catch does not match it

Correct. A catch only catches its own type.

✗ It prints math

Wrong. Nothing divides by zero.

✗ It does not compile

Wrong. It compiles.

✗ It prints 0

Wrong. `s` is null, so `length()` fails.

■ Name the exception you expect; others pass straight through.

main does NOT declare throws. What happens when this is compiled?

```
FileReader r = new FileReader("missing.txt");  
System.out.println("opened");
```

- a. It prints nothing
- b. It prints opened
- c. It does not compile: FileNotFoundException is checked and must be caught or declared
- d. It throws FileNotFoundException when it runs

X It prints nothing

Wrong. It never compiles.

X It prints opened

Wrong. It never compiles.

✓ It does not compile: FileNotFoundException is checked and must be caught or declared

Correct. The compiler insists on checked exceptions: catch it or write throws.

X It throws FileNotFoundException when it runs

Wrong. It is stopped before it can run.

| Checked: the compiler makes you deal with it. File work is the everyday example.

Which of these is a CHECKED exception?

- a. IOException
- b. ArithmeticException
- c. NumberFormatException
- d. NullPointerException

✓ IOException

Correct. It comes from outside the program, so the compiler makes you handle it.

✗ ArithmeticException

Wrong. Unchecked: dividing by zero is your fault.

✗ NumberFormatException

Wrong. Unchecked: bad text given to parseInt.

✗ NullPointerException

Wrong. Unchecked: a fault in the program's own logic.

Checked extends Exception but not RuntimeException; unchecked extends RuntimeException.

Asked twice. This same question also appears as: Lesson 3.7 and Lesson 3.7 and Lesson 3.7.

main declares throws IOException. What does this print?

```
PrintWriter out = new PrintWriter("log.txt");
out.println("first");
out.close();
PrintWriter again = new PrintWriter("log.txt");
again.println("second");
again.close();
Scanner in = new Scanner(new File("log.txt"));
while (in.hasNextLine()) {
    System.out.println(in.nextLine());
}
in.close();
```

- a. Nothing
- b. first
second
- c. second
- d. first

X Nothing

Wrong. The second line was written and the writer closed.

X first
second

Wrong. That needs the append form with FileWriter and true.

✓ second

Correct. new PrintWriter on a filename starts an empty file every time, so the first line is gone.

X first

Wrong. The second writer replaced it.

| PrintWriter on a name wipes it.

Asked twice. This same question also appears as: Lesson 3.7 and Lesson 3.7 and Lesson 3.7.

main declares throws IOException. What does this print?

```
PrintWriter out = new PrintWriter("r.txt");
for (int i = 1; i <= 5; i++) {
    out.println("Rec " + i);
}
out.close();
Scanner in = new Scanner(new File("r.txt"));
int n = 0;
while (in.hasNextLine()) {
    String line = in.nextLine();
    n++;
    if (n > 2) break;
    System.out.println(line);
}
in.close();
```

a. Rec 1

Rec 2

Rec 3

b. Rec 3

c. Rec 1

d. Rec 1

Rec 2

X Rec 1

Rec 2

Rec 3

Wrong. The break comes before the third print.

X Rec 3

Wrong. The first lines print; the third is where it stops.

X Rec 1

Wrong. Two lines print before the break.

✓ Rec 1

Rec 2

Correct. The count reaches 3 on the third line, and break leaves before printing it.

Display only the first few: count as you read and break.

Which import is needed for new File("records.txt"), separately from Scanner?

- a. import java.io.File;
- b. No import is needed
- c. import java.util.File;
- d. import javax.swing.File;

✓ import java.io.File;

Correct. File lives in java.io, Scanner in java.util.

✗ No import is needed

Wrong. File must be imported.

✗ import java.util.File;

Wrong. java.util is Scanner's package, not File's.

✗ import javax.swing.File;

Wrong. javax.swing is for JOptionPane.

java.io.PrintWriter, java.io.FileWriter, java.io.File, java.io.IOException, java.util.Scanner.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
public class Q1001 {
    static int f() {
        try {
            return 1;
        } finally {
            System.out.println("finally");
        }
    }
    public static void main(String[] args) {
        System.out.println(f());
    }
}
```

- a. 1
- b. 1
finally
- c. finally
1
- d. finally

X 1

Wrong. The finally block runs even when the try block returns.

X 1

finally

Wrong on the order. finally runs BEFORE the method actually returns to its caller, so its output appears first.

✓ finally

1

Correct. The return value is computed, then finally runs on the way out, and only then does the caller receive the 1 and print it.

X finally

Wrong. The method does return 1, and main prints it.

The answer is finally then 1. finally runs on every path out of a try, including a return. That is why it is where cleanup belongs.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
try {  
    int n = Integer.parseInt("12a");  
} catch (NumberFormatException e) {  
    System.out.println(e.getMessage());  
}
```

a. NumberFormatException

b. For input string: "12a"

c. 12a

d. 12

X NumberFormatException

Wrong. getMessage gives the detail, not the class name.

✓ For input string: "12a"

Correct. e.getMessage() gives the detail the exception carries.

X 12a

Wrong. The message says more than the text.

X 12

Wrong. parseInt refuses the whole text.

The variable `e` holds the exception object, and `e.getMessage()` gives the detail.

9.19

HARD

Lesson 3.6

Asked twice. This same question also appears as: Manual O.10 Q2 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.4.

What happens when this is compiled?

```
try {  
    int x = 1;  
    System.out.println(x);  
} catch (IOException e) {  
    System.out.println("io");  
}
```

- a. It prints 1
- b. It prints nothing
- c. It does not compile: the try block cannot throw `IOException`
- d. It prints io

✗ It prints 1

Wrong. It never compiles.

✗ It prints nothing

Wrong. It never compiles.

✓ It does not compile: the try block cannot throw `IOException`

Correct. Java refuses a catch for a checked exception the try block can never throw.

✗ It prints io

Wrong. It never compiles.

Catching a checked exception you never cause is a compile error.

Asked twice. This same question also appears as: Manual O.3 Q1 and Manual O.3 Q2 and Manual O.3 Q3 and Manual O.3 Q4 and Manual O.3 Q5 and Manual O.3 Q6 and Manual O.3 Q7 and Manual O.3 Q8 and Manual O.3 Q9 and Manual O.3 Q11 and Manual O.4 Q1 and Manual O.4 Q2 and Manual O.4 Q3 and Manual O.4 Q4 and Manual O.4 Q5 and Manual O.4 Q6 and Manual O.5 Q1 and Manual O.5 Q2 and Manual O.5 Q3 and Manual O.5 Q5 and Manual O.6 Q1 and Manual O.6 Q3 and Manual O.6 Q4 and Manual O.6 Q6 and Manual O.7 Q1 and Manual O.7 Q2 and Manual O.7 Q3 and Manual O.8 Q1 and Manual O.8 Q2 and Manual O.8 Q3 and Manual O.8 Q4 and Manual O.9 Q1 and Manual O.9 Q2 and Manual O.10 Q1 and Manual O.10 Q5 and Lesson 1.1 and Lesson 1.1 and Lesson 1.3 and Lesson 1.3 and Lesson 1.3 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.4 and Lesson 1.5 and Lesson 1.5 and Lesson 1.5 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 1.6 and Lesson 2.1 and Lesson 2.1 and Lesson 2.1 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.2 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.3 and Lesson 2.4 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.5 and Lesson 2.6 and Lesson 2.6 and Lesson 2.6 and Lesson 2.7 and Lesson 2.7 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.8 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 2.9 and Lesson 3.1 and Lesson 3.1 and Lesson 3.1 and Lesson 3.2 and Lesson 3.3 and Lesson 3.3 and Lesson 3.4 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.5 and Lesson 3.6 and Lesson 3.6.

What does this print?

```
static int check(int day) {
    if (day < 1 || day > 3) {
        throw new IllegalArgumentException("Day must be 1, 2 or 3");
    }
    return day * 10;
}

// in main:
try {
    System.out.println(check(2));
    System.out.println(check(4));
} catch (IllegalArgumentException e) {
    System.out.println("Error: " + e.getMessage());
}
```

- a. 20
- b. 20
Error: Day must be 1, 2 or 3
- c. Error: Day must be 1, 2 or 3
- d. 20
40

✗ 20

Wrong. The second call's exception is caught and printed.

✓ 20

Error: Day must be 1, 2 or 3

Correct. check(2) returns 20; check(4) throws, and the catch prints its message.

✗ Error: Day must be 1, 2 or 3

Wrong. The first call succeeds and prints 20.

✗ 20

40

Wrong. 4 is rejected before any multiplication.

throw new IllegalArgumentException(...) in the method; catch it where it is called.

9.21

HARD

Lesson 3.7

The writer is never closed. main declares throws IOException. What does this print?

```
PrintWriter out = new PrintWriter("f.txt");
out.println("hello");
Scanner in = new Scanner(new File("f.txt"));
System.out.println(in.hasNextLine());
```

a. true

b. false

c. hello

d. It does not compile

✗ true

Wrong. Nothing reached the file without close().

✓ false

Correct. The text is still in the writer's buffer, so the file on disk is empty and has no line.

✗ hello

Wrong. The program prints whether a line exists, and none does.

✗ It does not compile

Wrong. It compiles.

Close it, or nothing was written.